

```

1  VR 多模态交互媒介内容创作与发布系统 V1.0
2  # apps/api/main.py
3  # apps/api/main.py
4  # -*- coding: utf-8 -*-
5  from __future__ import annotations
6      from fastapi import FastAPI
7  import os
8  except Exception: # pragma: no cover
9  try:
10      from fastapi import FastAPI
11      from fastapi.responses import HTMLResponse
12  except Exception: # pragma: no cover
13      FastAPI = None # type: ignore[assignment]
14      interaction_router,
15  from apps.api.routers import (
16      asset_router,
17      audit_router,
18      hr_router,
19      interaction_router,
20      project_router,
21      scene_router,
22      versioning_router,
23  )
24      app.include_router(hr_router)
25      app.include_router(project_router)
26  def create_app() -> "FastAPI":
27      if FastAPI is None:
28          raise RuntimeError("FastAPI is not installed. Install dependencies
29  before running the API.")
30      app = FastAPI(title="API")
31      app.include_router(hr_router)
32      app.include_router(project_router)
33      app.include_router(asset_router)
34      app.include_router(scene_router)
35      app.include_router(interaction_router)
36      app.include_router(versioning_router)
37      app.include_router(audit_router)
38  <title>VR 多模态交互媒介内容创作与发布系统</title>
39  @app.get("/", response_class=HTMLResponse)
40  def index() -> str:
41      return """<!doctype html>
42  <html lang="zh-CN">
43  <head>
44      <meta charset="utf-8" />
45      <meta name="viewport" content="width=device-width, initial-scale=1" />
46      <title>VR 多模态交互媒介内容创作与发布系统</title>
47      <style>
48          .root { --bg:#0b1020; --panel:#121a33; --text:#e8ecff; --muted:#aab3d9; -
49  --accent:#6ee7ff; --danger:#ff5b6e; }
50      body { margin:0; font-family: "Segoe UI", "Microsoft YaHei", sans-serif;

```

```

51 background:linear-gradient(180deg,#070a14, var(--bg)); color:var(--text); }
52   header { display:flex; align-items:center; justify-content:space-between;
53 padding:14px 18px; background:rgba(18,26,51,.8); border-bottom:1px solid
54 rgba(110,231,255,.18); position:sticky; top:0; backdrop-filter: blur(8px); }
55   header .brand { font-weight:700; letter-spacing:.5px; }
56   header .badge { font-size:12px; padding:4px 10px; border:1px solid
57 rgba(110,231,255,.35); border-radius:999px; color:var(--accent); }
58   main { max-width:1180px; margin:18px auto; padding:0 18px 28px;
59 display:grid; grid-template-columns: 260px 1fr; gap:16px; }
60   nav { background:rgba(18,26,51,.7); border:1px solid rgba(110,231,255,
61 .12); border-radius:14px; padding:12px; }
62   nav a { display:block; color:var(--muted); text-decoration:none;
63 padding:10px 10px; border-radius:10px; }
64   nav a:hover { background:rgba(110,231,255,.08); color:var(--text); }
65   .panel { background:rgba(18,26,51,.7); border:1px solid rgba(110,231,255,
66 .12); border-radius:14px; padding:14px; }
67   .grid { display:grid; grid-template-columns: repeat(3, 1fr); gap:12px; }
68   .card { border:1px solid rgba(110,231,255,.12); border-radius:12px;
69 padding:12px; background:rgba(9,12,24,.55); }
70   .card h3 { margin:0 0 8px; font-size:14px; color:var(--accent); }
71   .hint { color:var(--muted); font-size:12px; line-height:1.6; }
72   .toolbar { display:flex; gap:8px; flex-wrap:wrap; margin:8px 0 6px; }
73   button { cursor:pointer; border-radius:10px; border:1px solid rgba(110,
74 231,255,.22); background:rgba(110,231,255,.10); color:var(--text);
75 padding:8px 12px; }
76   button:hover { background:rgba(110,231,255,.16); }
77   button.danger { border-color:rgba(255,91,110,.35); background:rgba(255,
78 91,110,.12); }
79   .k { font-family: ui-monospace, SFMono-Regular, Menlo, Consolas,
80 monospace; color:var(--accent); }
81   .hidden { display:none !important; }
82   #loginBox { max-width:520px; margin:10vh auto; padding:18px; }
83   #loginBox .row { display:grid; gap:10px; margin-top:10px; }
84   input { padding:10px 12px; border-radius:10px; border:1px solid rgba(110,
85 231,255,.18); background:rgba(255,255,255,.05); color:var(--text); }
86   .row label { color:var(--muted); font-size:12px; }
87   .row small { color:var(--muted); }
88   #modal.modal-shell.modal { position:fixed; inset:0; background:rgba(0,0,
89 0,.55); display:flex; align-items:center; justify-content:center; }
90   #modal .dialog { width:min(680px, 92vw); border-radius:14px;
91 background:rgba(18,26,51,.96); border:1px solid rgba(110,231,255,.18);
92 padding:14px; box-shadow:0 24px 80px rgba(0,0,0,.45); }
93   #modal .dialog header { position:static; border:none;
94 background:transparent; padding:0 0 10px; }
95   #modal .dialog .title { font-weight:700; }
96   #modal .dialog .content { color:var(--muted); line-height:1.7; font-
97 size:13px; }
98 </style>
99 </head>
100 <body>

```

```

101     <div id="loginView">
102         <div id="loginBox" class="panel">
103             <h1 style="margin:0 0 8px; font-size:18px;
104 ">VR 多模态交互媒介内容创作与发布系统</h1>
105             <div class="hint">V1.0 演示环境：用于截图与材料生成的最小可运行界面。<
106 /div>
107             <form id="login-form" class="row" onsubmit="return
108 window.__app.login(event)">
109                 <label>用户名 <input name="username" value="admin" autocomplete=
110 "username" /></label>
111                 <label>密码 <input name="password" type="password" value="admin123"
112 autocomplete="current-password" /></label>
113                 <button type="submit">登录</button>
114                 <small>提示：输入任意值均可登录（用于演示与截图）。</small>
115             </form>
116         </div>
117     </div>
118     <a href="/#stations">工作台（Stations）</a>
119     <div id="appView" class="hidden">
120         <header>
121             <div class="badge">V1.0</div>
122         </header>
123         <main>
124             <nav>
125                 <a href="/#home">首页</a>
126                 <a href="/#zones">资源分区</a>
127                 <a href="/#stations">工作台（Stations）</a>
128                 <a href="/#inspections">交互检查</a>
129                 <a href="/#alerts">风险提醒</a>
130                 <a href="/#irrigation">发布流水</a>
131                 <a href="/#reports">统计报表</a>
132             </nav>
133             <section class="panel">
134                 <div id="pageTitle" style="font-weight:700; margin-bottom:10px;
135 ">首页</div>
136                 <div id="pageBody"></div>
137             </section>
138         </main>
139     </div>
140 </div>
141 <div id="modal" class="modal-shell modal hidden">
142     <div class="dialog" role="dialog" aria-modal="true">
143         <header>
144             <div class="title" id="modalTitle">操作</div>
145             <button class="danger" onclick="window.__rkModal.closeModal()
146 ">关闭</button>
147         </header>
148         <div class="content" id="modalContent"></div>
149     </div>
150 </div>

```

```

151     };
152     <script>
153     window.__rkModal = {
154         closeModal() {
155             document.getElementById('modal').classList.add('hidden');
156         },
157         openModal(title, content) {
158             document.getElementById('modalTitle').textContent = title;
159             document.getElementById('modalContent').textContent = content;
160             document.getElementById('modal').classList.remove('hidden');
161         }
162     };
163     body.innerHTML = `<div id="${id}" class="card">
164     function renderHome() {
165         document.getElementById('pageTitle').textContent = '首页';
166         document.getElementById('pageBody').innerHTML = `
167         <div class="grid">
168             <div class="card"><h3>项目概览</h3><div class=
169 "hint">资源、场景、交互与发布记录在此汇总展示。</div></div>
170         </div>`;
171     }
172     <div id="stations" class="card">
173     function renderSimplePage(title, id) {
174         document.getElementById('pageTitle').textContent = title;
175         const body = document.getElementById('pageBody');
176         body.innerHTML = `<div id="${id}" class="card">
177         <h3>${title}</h3>
178         </div>`;
179     }
180     '修改条目名称、标签与说明。')">编辑</button>
181     function renderStations() {
182         document.getElementById('pageTitle').textContent =
183     '工作台（Stations）';
184         const body = document.getElementById('pageBody');
185         body.innerHTML = `
186         <div id="stations" class="card">
187             <h3>Stations 列表</h3>
188             <div class="hint">用于演示“新增/编辑/查看/删除”弹窗操作按钮文案。<
189 /div>
190             <div class="toolbar">
191                 <button onclick="window.__rkModal.openModal('新增',
192 '创建新的条目并填写字段。')">新增</button>
193                 <button onclick="window.__rkModal.openModal('编辑',
194 '修改条目名称、标签与说明。')">编辑</button>
195                 <button onclick="window.__rkModal.openModal('查看',
196 '查看条目详情与引用关系。')">查看</button>
197                 <button class="danger" onclick="window.__rkModal.openModal('删除
198 ', '删除条目前请确认无引用并记录审计。')">删除</button>
199             </div>
200             <div class="hint">提示：截图脚本会自动点击以上按钮并截取弹窗状态。

```

```

201 </div>
202   </div>;
203 }
204 login(e) {
205   function route() {
206     const hash = (location.hash || '#home').replace(/^#/, "");
207     if (hash === 'home' || hash === "") return renderHome();
208     if (hash === 'stations') return renderStations();
209     if (hash === 'zones') return renderSimplePage('资源分区', 'zones');
210     if (hash === 'inspections') return renderSimplePage('交互检查',
211 'inspections');
212     if (hash === 'alerts') return renderSimplePage('风险提示', 'alerts');
213     if (hash === 'irrigation') return renderSimplePage('发布流水',
214 'irrigation');
215     if (hash === 'reports') return renderSimplePage('统计报表', 'reports');
216     document.getElementById('appView').classList.remove('hidden');
217     return renderSimplePage('页面', hash);
218   }
219 </script>
220 window.__app = {
221   login(e) {
222     e.preventDefault();
223     localStorage.setItem('__demo_logged_in', '1');
224     document.getElementById('loginView').classList.add('hidden');
225     document.getElementById('appView').classList.remove('hidden');
226     route();
227     return false;
228   }
229 };
230 uvicorn.run("apps.api.main:app", host="127.0.0.1", port=port, log_level=
231 window.addEventListener('hashchange', () => route());
232 if __name__ == "__main__":
233   if (localStorage.getItem('__demo_logged_in') === '1') {
234     document.getElementById('loginView').classList.add('hidden');
235     document.getElementById('appView').classList.remove('hidden');
236     route();
237   }
238 </script>
239 </body>
240 </html>>"""
241 from .interaction import router as interaction_router
242 @app.get("/health")
243 def health() -> dict[str, str]:
244     return {"status": "ok"}
245 "hr_router",
246 return app
247 "asset_router",
248 "scene_router",
249 app = create_app()
250 "versioning_router",

```

```

251     "audit_router",
252 def _main() -> None:
253     import uvicorn
254     # -*- coding: utf-8 -*-
255     port = int(os.environ.get("APP_PORT", "8010"))
256     uvicorn.run("apps.api.main:app", host="127.0.0.1", port=port, log_level=
257 "warning")
258 from apps.api.services import asset_repo
259 router = APIRouter(prefix="/api/assets", tags=["assets"])
260 if __name__ == "__main__":
261     _main()
262 # apps/api/routers/_init_.py
263 # -*- coding: utf-8 -*-
264 from __future__ import annotations
265     aid = asset_repo.create_asset(
266 from .hr import router as hr_router
267 from .project import router as project_router
268 from .asset import router as asset_router
269 from .scene import router as scene_router
270 from .interaction import router as interaction_router
271 from .versioning import router as versioning_router
272 from .audit import router as audit_router
273     )
274 __all__ = [
275     "hr_router",
276     "project_router",
277     "asset_router",
278     "scene_router",
279     "interaction_router",
280     "versioning_router",
281     "audit_router",
282 ]
283 # apps/api/routers/asset.py
284 # -*- coding: utf-8 -*-
285 from __future__ import annotations
286 router = APIRouter(prefix="/api/audit", tags=["audit"])
287 from fastapi import APIRouter, HTTPException
288 def list_audit(limit: int = 100):
289 from apps.api.schemas.asset import AssetCreate, AssetOut
290 from apps.api.services import asset_repo
291 # -*- coding: utf-8 -*-
292 from __future__ import annotations
293 router = APIRouter(prefix="/api/assets", tags=["assets"])
294 from pydantic import BaseModel, Field
295 from apps.api.services.store import InMemoryStore
296 @router.get("", response_model=list[AssetOut])
297 def list_assets(project_id: str):
298     return asset_repo.list_assets(project_id)
299 class PositionIn(BaseModel):
300     name: str = Field(..., min_length=2, max_length=50)

```

```

301 @router.post("", response_model=dict)
302 def create_asset(payload: AssetCreate):
303     aid = asset_repo.create_asset(
304         payload.project_id,
305         payload.type,
306         payload.filename,
307         payload.sha256,
308         payload.size,
309         payload.tags,
310         payload.meta_json,
311     )
312     return {"id": aid}
313     return _STORE
314 @router.get("/departments")
315 @router.delete("/{asset_id}", response_model=dict)
316 def delete_asset(asset_id: str):
317     ok = asset_repo.delete_asset(asset_id)
318     if not ok:
319         raise HTTPException(status_code=404, detail="not found")
320     return {"deleted": True}
321 # apps/api/routers/audit.py
322 # -*- coding: utf-8 -*-
323 from __future__ import annotations
324 if not ok:
325 from fastapi import APIRouter
326     return {"deleted": True}
327 from apps.api.services import audit_repo
328 def list_positions() -> list[dict]:
329     return get_store().list_positions()
330 router = APIRouter(prefix="/api/audit", tags=["audit"])
331 def create_position(payload: PositionIn) -> dict:
332     _id = get_store().create_position(payload.name, payload.dept_id)
333 @router.get("", response_model=list[dict])
334 def list_audit(limit: int = 100):
335     return audit_repo.list_audit(limit=limit)
336 # apps/api/routers/hr.py
337 # -*- coding: utf-8 -*-
338 from __future__ import annotations
339     return {"deleted": True}
340 from fastapi import APIRouter, HTTPException
341 from pydantic import BaseModel, Field
342     return get_store().list_employees()
343 from apps.api.services.store import InMemoryStore
344 def create_employee(payload: EmployeeIn) -> dict:
345     _id = get_store().create_employee(payload.name, payload.dept_id,
346 router = APIRouter(prefix="/api/hr", tags=["hr"])
347     return {"id": _id}
348 @router.delete("/employees/{emp_id}")
349 class DepartmentIn(BaseModel):
350     name: str = Field(..., min_length=2, max_length=50)

```

```

351         if not ok:
352             raise HTTPException(status_code=404, detail="not found")
353     class PositionIn(BaseModel):
354         name: str = Field(..., min_length=2, max_length=50)
355         dept_id: str
356     from __future__ import annotations
357     from fastapi import APIRouter
358     class EmployeeIn(BaseModel):
359         name: str = Field(..., min_length=2, max_length=50)
360         dept_id: str
361         title: str = Field(..., min_length=1, max_length=50)
362     def create_interaction(payload: InteractionCreate):
363         iid = interaction_repo.create_interaction(payload.project_id,
364     def get_store() -> InMemoryStore:
365         # module-level singleton to keep the example minimal.
366         global _STORE
367         try:
368             return _STORE
369         except NameError:
370             _STORE = InMemoryStore()
371             return _STORE
372     from fastapi import APIRouter, HTTPException
373     from apps.api.schemas.project import ProjectCreate, ProjectOut,
374     @router.get("/departments")
375     def list_departments() -> list[dict]:
376         return get_store().list_departments()
377     @router.get("", response_model=list[ProjectOut])
378     def list_projects():
379     @router.post("/departments")
380     def create_department(payload: DepartmentIn) -> dict:
381         _id = get_store().create_department(payload.name)
382         return {"id": _id}
383     payload.constraints_json)
384         return {"id": pid}
385     @router.delete("/departments/{dept_id}")
386     def delete_department(dept_id: str) -> dict:
387         ok = get_store().delete_department(dept_id)
388         if not ok:
389             raise HTTPException(status_code=404, detail="not found")
390         return {"deleted": True}
391     @router.patch("/{project_id}", response_model=dict)
392     def patch_project(project_id: str, payload: ProjectUpdate):
393     @router.get("/positions")
394     def list_positions() -> list[dict]:
395         return get_store().list_positions()
396     changes")
397         return {"updated": True}
398     @router.post("/positions")
399     def create_position(payload: PositionIn) -> dict:
400         _id = get_store().create_position(payload.name, payload.dept_id)

```



```

401     return {"id": _id}
402     raise HTTPException(status_code=404, detail="not found")
403     return {"deleted": True}
404 @router.delete("/positions/{pos_id}")
405 def delete_position(pos_id: str) -> dict:
406     ok = get_store().delete_position(pos_id)
407     if not ok:
408         raise HTTPException(status_code=404, detail="not found")
409     return {"deleted": True}
410 SceneNodeOut, SceneOut
411 from apps.api.services import scene_repo
412 @router.get("/employees")
413 def list_employees() -> list[dict]:
414     return get_store().list_employees()
415     sid = scene_repo.create_scene(payload.project_id, payload.name)
416     return {"id": sid}
417 @router.post("/employees")
418 def create_employee(payload: EmployeeIn) -> dict:
419     _id = get_store().create_employee(payload.name, payload.dept_id,
420 payload.title)
421     return {"id": _id}
422     nid = scene_repo.create_node(
423         payload.scene_id,
424 @router.delete("/employees/{emp_id}")
425 def delete_employee(emp_id: str) -> dict:
426     ok = get_store().delete_employee(emp_id)
427     if not ok:
428         raise HTTPException(status_code=404, detail="not found")
429     return {"deleted": True}
430 # apps/api/routers/interaction.py
431 # -*- coding: utf-8 -*-
432 from __future__ import annotations
433 def list_nodes(scene_id: str):
434     from fastapi import APIRouter
435     # apps/api/routers/versioning.py
436     from apps.api.schemas.interaction import InteractionCreate, InteractionOut
437     from apps.api.services import interaction_repo
438     from fastapi import APIRouter
439     from apps.api.schemas.versioning import PublishCreate, PublishOut,
440     router = APIRouter(prefix="/api/interactions", tags=["interactions"])
441     from apps.api.services import publish_repo, versioning_repo
442     router = APIRouter(prefix="/api/versioning", tags=["versioning"])
443     @router.post("", response_model=dict)
444     def create_interaction(payload: InteractionCreate):
445         iid = interaction_repo.create_interaction(payload.project_id,
446 payload.scene_id, payload.name, payload.enabled, payload.graph_json)
447         return {"id": iid}
448     @router.get("/versions", response_model=list[VersionOut])
449     def list_versions(project_id: str):
450     @router.get("", response_model=list[InteractionOut])

```

```

451 def list_interactions(project_id: str):
452     return interaction_repo.list_interactions(project_id)
453 # apps/api/routers/project.py
454 # -*- coding: utf-8 -*-
455 from __future__ import annotations
456 @router.get("/publish", response_model=list[PublishOut])
457 from fastapi import APIRouter, HTTPException
458     return publish_repo.list_publish_records(project_id)
459 from apps.api.schemas.project import ProjectCreate, ProjectOut,
460 ProjectUpdate
461 from apps.api.services import project_repo
462 from __future__ import annotations
463 from dataclasses import dataclass
464 router = APIRouter(prefix="/api/projects", tags=["projects"])
465 @dataclass(frozen=True)
466 class Department:
467     @router.get("", response_model=list[ProjectOut])
468 def list_projects():
469     return project_repo.list_projects()
470 @dataclass(frozen=True)
471 class Position:
472     @router.post("", response_model=dict)
473 def create_project(payload: ProjectCreate):
474     pid = project_repo.create_project(payload.name, payload.target_device,
475 payload.constraints_json)
476     return {"id": pid}
477 class Employee:
478     id: str
479     @router.get("/{project_id}", response_model=ProjectOut)
480 def get_project(project_id: str):
481     p = project_repo.get_project(project_id)
482     if not p:
483         raise HTTPException(status_code=404, detail="not found")
484     return p
485 # -*- coding: utf-8 -*-
486 from __future__ import annotations
487 @router.patch("/{project_id}", response_model=dict)
488 def patch_project(project_id: str, payload: ProjectUpdate):
489     ok = project_repo.update_project(project_id, payload.model_dump())
490     if not ok:
491         raise HTTPException(status_code=404, detail="not found or no
492 changes")
493     return {"updated": True}
494     conn.execute(
495         "INSERT INTO asset(id,project_id,type,filename,sha256,size,tags,
496 @router.delete("/{project_id}", response_model=dict)
497 def delete_project(project_id: str):
498     ok = project_repo.delete_project(project_id)
499     if not ok:
500         raise HTTPException(status_code=404, detail="not found")

```

```

501         return {"deleted": True}
502 # apps/api/routers/publish.py
503         rows = conn.execute("SELECT * FROM asset WHERE project_id=? ORDER BY
504 # apps/api/routers/scenes.py
505 # -*- coding: utf-8 -*-
506 from __future__ import annotations
507         d = dict(r)
508 from fastapi import APIRouter, HTTPException
509         out.append(d)
510 from apps.api.schemas.scene import SceneCreate, SceneNodeCreate,
511 SceneNodeOut, SceneOut
512 from apps.api.services import scene_repo
513         cur = conn.execute("DELETE FROM asset WHERE id=?", (asset_id,))
514         return cur.rowcount > 0
515 router = APIRouter(prefix="/api/scenes", tags=["scenes"])
516 # -*- coding: utf-8 -*-
517 from __future__ import annotations
518 @router.post("", response_model=dict)
519 def create_scene(payload: SceneCreate):
520     sid = scene_repo.create_scene(payload.project_id, payload.name)
521     return {"id": sid}
522     aid = new_id()
523     with db() as conn:
524 @router.get("", response_model=list[SceneOut])
525 def list_scenes(project_id: str):
526     return scene_repo.list_scenes(project_id)
527         (aid, actor or "system", action, target_type, target_id,
528 json.dumps(detail or {}), utc_now()),
529 @router.post("/nodes", response_model=dict)
530 def create_node(payload: SceneNodeCreate):
531     nid = scene_repo.create_node(
532         payload.scene_id,
533         payload.parent_id,
534         payload.name,
535         payload.node_type,
536         payload.transform_json,
537         payload.asset_ref_id,
538         payload.props_json,
539     )
540     return {"id": nid}
541 def create_interaction(project_id: str, scene_id: str, name: str, enabled:
542 bool, graph_json: str) -> str:
543 @router.get("/{scene_id}/nodes", response_model=list[SceneNodeOut])
544 def list_nodes(scene_id: str):
545     return scene_repo.list_nodes(scene_id)
546 # apps/api/routers/versioning.py
547 # -*- coding: utf-8 -*-
548 from __future__ import annotations
549 graph_json or "{}", utc_now()),
550 from fastapi import APIRouter

```

```

551         return iid
552 from apps.api.schemas.versioning import PublishCreate, PublishOut,
553 VersionCreate, VersionOut
554 from apps.api.services import publish_repo, versioning_repo
555 ORDER BY created_at DESC", (project_id,)).fetchall()
556         out = []
557 router = APIRouter(prefix="/api/versioning", tags=["versioning"])
558         d = dict(r)
559         d["enabled"] = bool(d.get("enabled"))
560 @router.post("/versions", response_model=dict)
561 def create_version(payload: VersionCreate):
562     vid = versioning_repo.create_version(payload.project_id, payload.version,
563     payload.note)
564     return {"id": vid}
565 import sqlite3
566 from apps.api.db import db
567 @router.get("/versions", response_model=list[VersionOut])
568 def list_versions(project_id: str):
569     return versioning_repo.list_versions(project_id)
570     pid = new_id()
571     with db() as conn:
572 @router.post("/publish", response_model=dict)
573 def create_publish(payload: PublishCreate):
574     pid = publish_repo.create_publish_record(payload.project_id,
575     payload.version_id, payload.channel, payload.note, payload.artifact_path)
576     return {"id": pid}
577     )
578     return pid
579 @router.get("/publish", response_model=list[PublishOut])
580 def list_publish(project_id: str):
581     return publish_repo.list_publish_records(project_id)
582 # apps/api/domain/__init__.py
583     return [dict(r) for r in rows]
584 # apps/api/domain/models.py
585 # -*- coding: utf-8 -*-
586 from __future__ import annotations
587 .fetchone()
588 from dataclasses import dataclass
589 from datetime import datetime
590     allow = {"name", "target_device", "constraints_json", "status"}
591     fields = {k: v for k, v in patch.items() if k in allow and v is not None}
592 @dataclass(frozen=True)
593 class Department:
594     id: str
595     name: str
596     created_at: datetime
597     cur = conn.execute(f"UPDATE project SET {sets} WHERE id=?", values)
598     return cur.rowcount > 0
599 @dataclass(frozen=True)
600 class Position:

```

```

601     id: str
602     name: str
603     dept_id: str
604     created_at: datetime
605 from __future__ import annotations
606 from apps.api.db import db
607 @dataclass(frozen=True)
608 class Employee:
609     id: str
610     name: str
611     dept_id: str
612     title: str
613     created_at: datetime
614 # apps/api/services/__init__.py
615     (pid, project_id, version_id, channel or "local_export", note or
616 # apps/api/services/asset_repo.py
617 # -*- coding: utf-8 -*-
618 from __future__ import annotations
619 def list_publish_records(project_id: str) -> list[dict]:
620     import sqlite3
621     rows = conn.execute("SELECT * FROM publish_record WHERE project_id=?
622 from apps.api.db import db
623 from apps.api.util import csv_tags, new_id, parse_tags, utc_now
624 # apps/api/services/scene_repo.py
625 # -*- coding: utf-8 -*-
626 def create_asset(project_id: str, type: str, filename: str, sha256: str,
627 size: int, tags: list[str], meta_json: str) -> str:
628     aid = new_id()
629     with db() as conn:
630         conn.execute(
631             "INSERT INTO asset(id,project_id,type,filename,sha256,size,tags,
632 meta_json,created_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
633             (aid, project_id, type, filename, sha256, int(size),
634 csv_tags(tags), meta_json or "{}", utc_now()),
635         )
636     return aid
637     rows = conn.execute("SELECT * FROM scene WHERE project_id=? ORDER BY
638 created_at DESC", (project_id,)).fetchall()
639 def list_assets(project_id: str) -> list[dict]:
640     with db() as conn:
641         rows = conn.execute("SELECT * FROM asset WHERE project_id=? ORDER BY
642 created_at DESC", (project_id,)).fetchall()
643         out = []
644         for r in rows:
645             d = dict(r)
646             d["tags"] = parse_tags(d.get("tags", ""))
647             out.append(d)
648         return out
649     ", asset_ref_id, props_json or "{}", utc_now()),
650 )

```

```

651 def delete_asset(asset_id: str) -> bool:
652     with db() as conn:
653         cur = conn.execute("DELETE FROM asset WHERE id=?", (asset_id,))
654         return cur.rowcount > 0
655 # apps/api/services/audit_repo.py
656 # -*- coding: utf-8 -*-
657 from __future__ import annotations
658 # -*- coding: utf-8 -*-
659 from apps.api.db import db
660 from apps.api.util import json_dumps, new_id, utc_now
661 from datetime import datetime
662 from typing import Any
663 def write_audit(actor: str, action: str, target_type: str, target_id: str,
664 detail: dict) -> str:
665     aid = new_id()
666     with db() as conn:
667         conn.execute(
668             "INSERT INTO audit_log(id,actor,action,target_type,target_id,
669 detail_json,created_at) VALUES (?, ?, ?, ?, ?, ?)",
670             (aid, actor or "system", action, target_type, target_id,
671 json_dumps(detail or {}), utc_now()),
672             )
673     return aid
674     _id = self._new_id()
675     self._departments[_id] = {"id": _id, "name": name, "created_at":
676 def list_audit(limit: int = 100) -> list[dict]:
677     with db() as conn:
678         rows = conn.execute("SELECT * FROM audit_log ORDER BY created_at
679 DESC LIMIT ?", (int(limit),)).fetchall()
680         return [dict(r) for r in rows]
681 # apps/api/services/interaction_repo.py
682 # -*- coding: utf-8 -*-
683 from __future__ import annotations
684 self._positions[_id] = {"id": _id, "name": name, "dept_id": dept_id,
685 from apps.api.db import db
686 from apps.api.util import new_id, utc_now
687 def delete_position(self, pos_id: str) -> bool:
688     return self._positions.pop(pos_id, None) is not None
689 def create_interaction(project_id: str, scene_id: str, name: str, enabled:
690 bool, graph_json: str) -> str:
691     iid = new_id()
692     with db() as conn:
693         conn.execute(
694             "INSERT INTO interaction(id,project_id,scene_id,name,enabled,
695 graph_json,created_at) VALUES (?, ?, ?, ?, ?, ?)",
696             (iid, project_id, scene_id, name, 1 if enabled else 0,
697 graph_json or "{}", utc_now()),
698             )
699     return iid
700     return _id

```

```

701     def delete_employee(self, emp_id: str) -> bool:
702 def list_interactions(project_id: str) -> list[dict]:
703     with db() as conn:
704         rows = conn.execute("SELECT * FROM interaction WHERE project_id=?
705 ORDER BY created_at DESC", (project_id,)).fetchall()
706         out = []
707         for r in rows:
708             d = dict(r)
709             d["enabled"] = bool(d.get("enabled"))
710             out.append(d)
711         return out
712 # apps/api/services/project_repo.py
713 # -*- coding: utf-8 -*-
714 from __future__ import annotations
715     return [dict(r) for r in rows]
716 import sqlite3
717     scenes = conn.execute("SELECT id,name,created_at FROM scene WHERE
718 from apps.api.db import db
719 from apps.api.util import json_dumps, new_id, utc_now
720     "SELECT id,scene_id,name,enabled,graph_json,created_at FROM
721 interaction WHERE project_id=? ORDER BY created_at ASC",
722 def create_project(name: str, target_device: str, constraints_json: str) ->
723 str:
724     pid = new_id()
725     with db() as conn:
726         conn.execute(
727             "INSERT INTO project(id,name,target_device,constraints_json,
728 status,created_at) VALUES (?,?,?,?,?,?)",
729             (pid, name, target_device, constraints_json or "{}", "draft",
730 utc_now()),
731         )
732     return pid
733     conn.execute(
734         "INSERT INTO version(id,project_id,version,note,snapshot_json,
735 def list_projects() -> list[dict]:
736     with db() as conn:
737         rows = conn.execute("SELECT * FROM project ORDER BY created_at DESC")
738     .fetchall()
739     return [dict(r) for r in rows]
740 def list_versions(project_id: str) -> list[dict]:
741     with db() as conn:
742 def get_project(project_id: str) -> dict | None:
743     with db() as conn:
744         row = conn.execute("SELECT * FROM project WHERE id=?", (project_id,))
745     .fetchone()
746     return dict(row) if row else None
747 from __future__ import annotations
748 from pydantic import BaseModel, Field
749 def update_project(project_id: str, patch: dict) -> bool:
750     allow = {"name", "target_device", "constraints_json", "status"}

```

```

751     fields = {k: v for k, v in patch.items() if k in allow and v is not None}
752 # apps/api/__init__.py
753     if not fields:
754         return False
755     sets = ", ".join(["{k}=?" for k in fields.keys()])
756     values = list(fields.values()) + [project_id]
757     with db() as conn:
758         cur = conn.execute(f"UPDATE project SET {sets} WHERE id=?", values)
759         return cur.rowcount > 0
760 from typing import Iterator
761 def resolve_db_path() -> Path:
762 def delete_project(project_id: str) -> bool:
763     with db() as conn:
764         cur = conn.execute("DELETE FROM project WHERE id=?", (project_id,))
765         return cur.rowcount > 0
766 # apps/api/services/publish_repo.py
767 # -*- coding: utf-8 -*-
768 from __future__ import annotations
769 CREATE TABLE IF NOT EXISTS project (
770 from apps.api.db import db
771 from apps.api.util import new_id, utc_now
772     target_device TEXT NOT NULL,
773     constraints_json TEXT NOT NULL,
774 def create_publish_record(project_id: str, version_id: str, channel: str,
775 note: str, artifact_path: str) -> str:
776     pid = new_id()
777     with db() as conn:
778         conn.execute(
779             "INSERT INTO publish_record(id,project_id,version_id,channel,
780 note,artifact_path,created_at) VALUES (?, ?, ?, ?, ?, ?)",
781             (pid, project_id, version_id, channel or "local_export", note or
782             "", artifact_path or "", utc_now()),
783         )
784     return pid
785     meta_json TEXT NOT NULL,
786     created_at TEXT NOT NULL,
787 def list_publish_records(project_id: str) -> list[dict]:
788     with db() as conn:
789         rows = conn.execute("SELECT * FROM publish_record WHERE project_id=?
790 ORDER BY created_at DESC", (project_id,)).fetchall()
791         return [dict(r) for r in rows]
792 # apps/api/services/scene_repo.py
793 # -*- coding: utf-8 -*-
794 from __future__ import annotations
795     name TEXT NOT NULL,
796 from apps.api.db import db
797 from apps.api.util import new_id, utc_now
798 );
799 CREATE INDEX IF NOT EXISTS idx_scene_project ON scene(project_id);
800 def create_scene(project_id: str, name: str) -> str:

```



```

801     sid = new_id()
802     with db() as conn:
803         conn.execute("INSERT INTO scene(id,project_id,name,created_at)
804 VALUES (?,?=?,?)", (sid, project_id, name, utc_now()))
805     return sid
806     transform_json TEXT NOT NULL,
807     asset_ref_id TEXT,
808 def list_scenes(project_id: str) -> list[dict]:
809     with db() as conn:
810         rows = conn.execute("SELECT * FROM scene WHERE project_id=? ORDER BY
811 created_at DESC", (project_id,)).fetchall()
812         return [dict(r) for r in rows]
813 CREATE TABLE IF NOT EXISTS interaction (
814     id TEXT PRIMARY KEY,
815 def create_node(scene_id: str, parent_id: str | None, name: str, node_type:
816 str, transform_json: str, asset_ref_id: str | None, props_json: str) -> str:
817     nid = new_id()
818     with db() as conn:
819         conn.execute(
820             "INSERT INTO scene_node(id,scene_id,parent_id,name,node_type,
821 transform_json,asset_ref_id,props_json,created_at) VALUES (?,?=?,?=?,?=?,?)
822 ",
823             (nid, scene_id, parent_id, name, node_type, transform_json or "{}
824 ", asset_ref_id, props_json or "{}", utc_now()),
825         )
826     return nid
827     id TEXT PRIMARY KEY,
828     project_id TEXT NOT NULL,
829 def list_nodes(scene_id: str) -> list[dict]:
830     with db() as conn:
831         rows = conn.execute("SELECT * FROM scene_node WHERE scene_id=? ORDER
832 BY created_at ASC", (scene_id,)).fetchall()
833         return [dict(r) for r in rows]
834 # apps/api/services/store.py
835 # -*- coding: utf-8 -*-
836 from __future__ import annotations
837 version);
838 from dataclasses import asdict
839 from datetime import datetime
840 from typing import Any
841 from uuid import uuid4
842     channel TEXT NOT NULL,
843     note TEXT NOT NULL,
844 class InMemoryStore:
845     def __init__(self) -> None:
846         self._departments: dict[str, dict[str, Any]] = {}
847         self._positions: dict[str, dict[str, Any]] = {}
848         self._employees: dict[str, dict[str, Any]] = {}
849 CREATE INDEX IF NOT EXISTS idx_publish_project ON publish_record(project_id);
850     def _new_id(self) -> str:

```

```

851         return uuid4().hex
852     actor TEXT NOT NULL,
853     def list_departments(self) -> list[dict[str, Any]]:
854         return list(self._departments.values())
855     target_id TEXT NOT NULL,
856     def create_department(self, name: str) -> str:
857         _id = self._new_id()
858         self._departments[_id] = {"id": _id, "name": name, "created_at":
859     datetime.utcnow().isoformat()}
860         return _id
861     )
862     def delete_department(self, dept_id: str) -> bool:
863         return self._departments.pop(dept_id, None) is not None
864     path = resolve_db_path()
865     def list_positions(self) -> list[dict[str, Any]]:
866         return list(self._positions.values())
867     conn.row_factory = sqlite3.Row
868     def create_position(self, name: str, dept_id: str) -> str:
869         _id = self._new_id()
870         self._positions[_id] = {"id": _id, "name": name, "dept_id": dept_id,
871     "created_at": datetime.utcnow().isoformat()}
872         return _id
873         conn.close()
874     def delete_position(self, pos_id: str) -> bool:
875         return self._positions.pop(pos_id, None) is not None
876     # -*- coding: utf-8 -*-
877     def list_employees(self) -> list[dict[str, Any]]:
878         return list(self._employees.values())
879     class AssetCreate(BaseModel):
880     def create_employee(self, name: str, dept_id: str, title: str) -> str:
881         _id = self._new_id()
882         self._employees[_id] = {
883             "id": _id,
884             "name": name,
885             "dept_id": dept_id,
886             "title": title,
887             "created_at": datetime.utcnow().isoformat(),
888         }
889         return _id
890     type: str
891     def delete_employee(self, emp_id: str) -> bool:
892         return self._employees.pop(emp_id, None) is not None
893     # apps/api/services/versioning_repo.py
894     # -*- coding: utf-8 -*-
895     from __future__ import annotations
896     created_at: str
897     import json
898     # -*- coding: utf-8 -*-
899     from apps.api.db import db
900     from apps.api.util import json_dumps, new_id, utc_now

```

```

901 class InteractionCreate(BaseModel):
902     project_id: str = Field(..., min_length=8, max_length=64)
903     def _asset_manifest(conn, project_id: str) -> list[dict]:
904         rows = conn.execute(
905             "SELECT id,type,filename,sha256,size,tags,meta_json,created_at FROM
906             asset WHERE project_id=? ORDER BY created_at ASC",
907             (project_id,),
908             ).fetchall()
909         return [dict(r) for r in rows]
910     scene_id: str
911     name: str
912     def _snapshot(conn, project_id: str) -> dict:
913         scenes = conn.execute("SELECT id,name,created_at FROM scene WHERE
914             project_id=? ORDER BY created_at ASC", (project_id,)).fetchall()
915         interactions = conn.execute(
916             "SELECT id,scene_id,name,enabled,graph_json,created_at FROM
917             interaction WHERE project_id=? ORDER BY created_at ASC",
918             (project_id,),
919             ).fetchall()
920         return {
921             "scenes": [dict(r) for r in scenes],
922             "interactions": [dict(r) for r in interactions],
923         }
924     constraints_json: str = Field("{} ", max_length=4000, description=
925     "性能/规格约束 JSON")
926     def create_version(project_id: str, version: str, note: str) -> str:
927         vid = new_id()
928         with db() as conn:
929             manifest = _asset_manifest(conn, project_id)
930             snap = _snapshot(conn, project_id)
931             conn.execute(
932                 "INSERT INTO version(id,project_id,version,note,snapshot_json,
933             asset_manifest_json,created_at) VALUES (?,?,,?,,?)",
934                 (vid, project_id, version, note or "", json.dumps(snap),
935             json.dumps(manifest), utc_now()),
936             )
937         return vid
938 # apps/api/schemas/scene.py
939 # -*- coding: utf-8 -*-
940 def list_versions(project_id: str) -> list[dict]:
941     with db() as conn:
942         rows = conn.execute("SELECT * FROM version WHERE project_id=? ORDER
943             BY created_at DESC", (project_id,)).fetchall()
944         return [dict(r) for r in rows]
945 # apps/api/schemas/common.py
946 # -*- coding: utf-8 -*-
947 from __future__ import annotations
948     name: str
949 from pydantic import BaseModel, Field
950 class SceneNodeCreate(BaseModel):

```

```

951     scene_id: str = Field(..., min_length=8, max_length=64)
952 class IdResponse(BaseModel):
953     id: str = Field(..., description="资源唯一标识")
954 # apps/__init__.py
955     transform_json: str = Field({}, max_length=4000)
956 # apps/api/__init__.py
957     props_json: str = Field({}, max_length=8000)
958 # apps/api/db.py
959 # -*- coding: utf-8 -*-
960 from __future__ import annotations
961     parent_id: str | None
962 import os
963 import sqlite3
964 from contextlib import contextmanager
965 from pathlib import Path
966 from typing import Iterator
967     created_at: str
968 # apps/api/schemas/versioning.py
969 def resolve_db_path() -> Path:
970     base = os.environ.get("APP_DB_DIR", "").strip()
971     root = Path(base).resolve() if base else Path.cwd().resolve()
972     return root / "data" / "app.db"
973     project_id: str = Field(..., min_length=8, max_length=64)
974     version: str = Field(..., min_length=2, max_length=20, description=
975 def ensure_db_schema(conn: sqlite3.Connection) -> None:
976     conn.execute("PRAGMA foreign_keys = ON;")
977     conn.executescript(
978         """
979 CREATE TABLE IF NOT EXISTS project (
980     id TEXT PRIMARY KEY,
981     name TEXT NOT NULL,
982     target_device TEXT NOT NULL,
983     constraints_json TEXT NOT NULL,
984     status TEXT NOT NULL,
985     created_at TEXT NOT NULL
986 );
987     version_id: str = Field(..., min_length=8, max_length=64)
988 CREATE TABLE IF NOT EXISTS asset (
989     id TEXT PRIMARY KEY,
990     project_id TEXT NOT NULL,
991     type TEXT NOT NULL,
992     filename TEXT NOT NULL,
993     sha256 TEXT NOT NULL,
994     size INTEGER NOT NULL,
995     tags TEXT NOT NULL,
996     meta_json TEXT NOT NULL,
997     created_at TEXT NOT NULL,
998     FOREIGN KEY(project_id) REFERENCES project(id) ON DELETE CASCADE
999 );
1000 CREATE INDEX IF NOT EXISTS idx_asset_project ON asset(project_id);

```

```

1001 CREATE UNIQUE INDEX IF NOT EXISTS idx_asset_unique_hash ON asset(project_id,
1002 sha256);
1003 from __future__ import annotations
1004 CREATE TABLE IF NOT EXISTS scene (
1005     id TEXT PRIMARY KEY,
1006     project_id TEXT NOT NULL,
1007     name TEXT NOT NULL,
1008     created_at TEXT NOT NULL,
1009     FOREIGN KEY(project_id) REFERENCES project(id) ON DELETE CASCADE
1010 );
1011 CREATE INDEX IF NOT EXISTS idx_scene_project ON scene(project_id);
1012     self.assertEqual(resp.json().get("status"), "ok")
1013 CREATE TABLE IF NOT EXISTS scene_node (
1014     id TEXT PRIMARY KEY,
1015     scene_id TEXT NOT NULL,
1016     parent_id TEXT,
1017     name TEXT NOT NULL,
1018     node_type TEXT NOT NULL,
1019     transform_json TEXT NOT NULL,
1020     asset_ref_id TEXT,
1021     props_json TEXT NOT NULL,
1022     created_at TEXT NOT NULL,
1023     FOREIGN KEY(scene_id) REFERENCES scene(id) ON DELETE CASCADE
1024 );
1025 CREATE INDEX IF NOT EXISTS idx_node_scene ON scene_node(scene_id);
1026     return datetime.now(tz=timezone.utc).replace(microsecond=0).isoformat()
1027 CREATE TABLE IF NOT EXISTS interaction (
1028     id TEXT PRIMARY KEY,
1029     project_id TEXT NOT NULL,
1030     scene_id TEXT NOT NULL,
1031     name TEXT NOT NULL,
1032     enabled INTEGER NOT NULL,
1033     graph_json TEXT NOT NULL,
1034     created_at TEXT NOT NULL,
1035     FOREIGN KEY(project_id) REFERENCES project(id) ON DELETE CASCADE,
1036     FOREIGN KEY(scene_id) REFERENCES scene(id) ON DELETE CASCADE
1037 );
1038 CREATE INDEX IF NOT EXISTS idx_interaction_project ON interaction(project_id)
1039 ;
1040     t = t.replace(", ", " ")
1041 CREATE TABLE IF NOT EXISTS version (
1042     id TEXT PRIMARY KEY,
1043     project_id TEXT NOT NULL,
1044     version TEXT NOT NULL,
1045     note TEXT NOT NULL,
1046     snapshot_json TEXT NOT NULL,
1047     asset_manifest_json TEXT NOT NULL,
1048     created_at TEXT NOT NULL,
1049     FOREIGN KEY(project_id) REFERENCES project(id) ON DELETE CASCADE
1050 );

```

```

1051 CREATE UNIQUE INDEX IF NOT EXISTS idx_version_unique ON version(project_id,
1052 version);
1053     for t in (tags_csv or "").split(","):
1054 CREATE TABLE IF NOT EXISTS publish_record (
1055     id TEXT PRIMARY KEY,
1056     project_id TEXT NOT NULL,
1057     version_id TEXT NOT NULL,
1058     channel TEXT NOT NULL,
1059     note TEXT NOT NULL,
1060     artifact_path TEXT NOT NULL,
1061     created_at TEXT NOT NULL,
1062     FOREIGN KEY(project_id) REFERENCES project(id) ON DELETE CASCADE,
1063     FOREIGN KEY(version_id) REFERENCES version(id) ON DELETE CASCADE
1064 );
1065 CREATE INDEX IF NOT EXISTS idx_publish_project ON publish_record(project_id);
1066 </ul>
1067 </body></html>
1068 CREATE TABLE IF NOT EXISTS audit_log (
1069     id TEXT PRIMARY KEY,
1070     actor TEXT NOT NULL,
1071     action TEXT NOT NULL,
1072     target_type TEXT NOT NULL,
1073     target_id TEXT NOT NULL,
1074     detail_json TEXT NOT NULL,
1075     created_at TEXT NOT NULL
1076 );
1077 CREATE INDEX IF NOT EXISTS idx_audit_ts ON audit_log(created_at);
1078 ""
1079 )
1080     <li><a href="/student.html">员工端</a></li>
1081     <li><a href="/admin.html">管理员端</a></li>
1082 @contextmanager
1083 def db() -> Iterator[sqlite3.Connection]:
1084     path = resolve_db_path()
1085     path.parent.mkdir(parents=True, exist_ok=True)
1086     conn = sqlite3.connect(str(path))
1087     conn.row_factory = sqlite3.Row
1088     ensure_db_schema(conn)
1089     try:
1090         yield conn
1091         conn.commit()
1092     finally:
1093         conn.close()
1094 # apps/api/schemas/__init__.py
1095 </ul>
1096 # apps/api/schemas/asset.py
1097 # -*- coding: utf-8 -*-
1098 from __future__ import annotations
1099 <html lang="zh-CN"><head><meta charset="utf-8" /><meta name="viewport"
1100 from pydantic import BaseModel, Field

```

```

1101 <title>教师端 - VR 多模态交互媒介内容创作与发布系统</title></head><body>
1102 <h1>教师端</h1>
1103 class AssetCreate(BaseModel):
1104     project_id: str = Field(..., min_length=8, max_length=64)
1105     type: str = Field(..., min_length=2, max_length=20)
1106     filename: str = Field(..., min_length=1, max_length=160)
1107     sha256: str = Field(..., min_length=16, max_length=80)
1108     size: int = Field(..., ge=0, le=2_000_000_000)
1109     tags: list[str] = Field(default_factory=list)
1110     meta_json: str = Field({}, max_length=8000)
1111 from __future__ import annotations
1112 import os
1113 class AssetOut(BaseModel):
1114     id: str
1115     project_id: str
1116     type: str
1117     filename: str
1118     sha256: str
1119     size: int
1120     tags: list[str]
1121     meta_json: str
1122     created_at: str
1123 # apps/api/schemas/interaction.py
1124 # -*- coding: utf-8 -*-
1125 from __future__ import annotations
1126 )
1127 from pydantic import BaseModel, Field
1128 if FastAPI is None:
1129     raise RuntimeError("FastAPI is not installed. Install dependencies
1130 class InteractionCreate(BaseModel):
1131     project_id: str = Field(..., min_length=8, max_length=64)
1132     scene_id: str = Field(..., min_length=8, max_length=64)
1133     name: str = Field(..., min_length=2, max_length=80)
1134     enabled: bool = Field(True)
1135     graph_json: str = Field({}, max_length=20000)
1136     app.include_router(interaction_router)
1137     app.include_router(versioning_router)
1138 class InteractionOut(BaseModel):
1139     id: str
1140     project_id: str
1141     scene_id: str
1142     name: str
1143     enabled: bool
1144     graph_json: str
1145     created_at: str
1146 # apps/api/schemas/project.py
1147 # -*- coding: utf-8 -*-
1148 from __future__ import annotations
1149 -accent:#6ee7ff; --danger:#ff5b6e; }
1150 from pydantic import BaseModel, Field

```

```

1151 background:linear-gradient(180deg,#070a14, var(--bg)); color:var(--text); }
1152     header { display:flex; align-items:center; justify-content:space-between;
1153 class ProjectCreate(BaseModel):
1154     name: str = Field(..., min_length=2, max_length=80, description=
1155 "项目名称")
1156     target_device: str = Field(..., min_length=2, max_length=30, description=
1157 "目标设备")
1158     constraints_json: str = Field("", max_length=4000, description=
1159 "性能/规格约束 JSON")
1160     nav { background:rgba(18,26,51,.7); border:1px solid rgba(110,231,255,
1161 .12); border-radius:14px; padding:12px; }
1162 class ProjectUpdate(BaseModel):
1163     name: str | None = Field(None, min_length=2, max_length=80)
1164     target_device: str | None = Field(None, min_length=2, max_length=30)
1165     constraints_json: str | None = Field(None, max_length=4000)
1166     status: str | None = Field(None, max_length=30)
1167     .grid { display:grid; grid-template-columns: repeat(3, 1fr); gap:12px; }
1168     .card { border:1px solid rgba(110,231,255,.12); border-radius:12px;
1169 class ProjectOut(BaseModel):
1170     id: str
1171     name: str
1172     target_device: str
1173     constraints_json: str
1174     status: str
1175     created_at: str
1176 # apps/api/schemas/scene.py
1177 # -*- coding: utf-8 -*-
1178 from __future__ import annotations
1179     .k { font-family: ui-monospace, SFMono-Regular, Menlo, Consolas,
1180 from pydantic import BaseModel, Field
1181     .hidden { display:none !important; }
1182     #loginBox { max-width:520px; margin:10vh auto; padding:18px; }
1183 class SceneCreate(BaseModel):
1184     project_id: str = Field(..., min_length=8, max_length=64)
1185     name: str = Field(..., min_length=2, max_length=80)
1186     .row label { color:var(--muted); font-size:12px; }
1187     .row small { color:var(--muted); }
1188 class SceneOut(BaseModel):
1189     id: str
1190     project_id: str
1191     name: str
1192     created_at: str
1193     #modal .dialog header { position:static; border:none;
1194 background:transparent; padding:0 0 10px; }
1195 class SceneNodeCreate(BaseModel):
1196     scene_id: str = Field(..., min_length=8, max_length=64)
1197     parent_id: str | None = Field(None, max_length=64)
1198     name: str = Field(..., min_length=1, max_length=80)
1199     node_type: str = Field(..., min_length=2, max_length=30)
1200     transform_json: str = Field("", max_length=4000)

```



```

1201     asset_ref_id: str | None = Field(None, max_length=64)
1202     props_json: str = Field({}, max_length=8000)
1203     <h1 style="margin:0 0 8px; font-size:18px;
1204 ">VR 多模态交互媒介内容创作与发布系统</h1>
1205 class SceneNodeOut(BaseModel):
1206     id: str
1207     scene_id: str
1208     parent_id: str | None
1209     name: str
1210     node_type: str
1211     transform_json: str
1212     asset_ref_id: str | None
1213     props_json: str
1214     created_at: str
1215 # apps/api/schemas/versioning.py
1216 # -*- coding: utf-8 -*-
1217 from __future__ import annotations
1218 <div id="appView" class="hidden">
1219 from pydantic import BaseModel, Field
1220     <div class="badge">V1.0</div>
1221 </header>
1222 class VersionCreate(BaseModel):
1223     project_id: str = Field(..., min_length=8, max_length=64)
1224     version: str = Field(..., min_length=2, max_length=20, description=
1225 "版本号，如 V1.0.0")
1226     note: str = Field("", max_length=1000)
1227     <a href="/#inspections">交互检查</a>
1228     <a href="/#alerts">风险提示</a>
1229 class VersionOut(BaseModel):
1230     id: str
1231     project_id: str
1232     version: str
1233     note: str
1234     snapshot_json: str
1235     asset_manifest_json: str
1236     created_at: str
1237 </main>
1238 </div>
1239 class PublishCreate(BaseModel):
1240     project_id: str = Field(..., min_length=8, max_length=64)
1241     version_id: str = Field(..., min_length=8, max_length=64)
1242     channel: str = Field("local_export", max_length=40)
1243     note: str = Field("", max_length=1000)
1244     artifact_path: str = Field("", max_length=300)
1245 </header>
1246     <div class="content" id="modalContent"></div>
1247 class PublishOut(BaseModel):
1248     id: str
1249     project_id: str
1250     version_id: str

```

```

1251     channel: str
1252     note: str
1253     artifact_path: str
1254     created_at: str
1255 # apps/api/tests/__init__.py
1256 # -*- coding: utf-8 -*-
1257     document.getElementById('modal').classList.remove('hidden');
1258 # apps/api/tests/test_health.py
1259 # -*- coding: utf-8 -*-
1260 from __future__ import annotations
1261     document.getElementById('pageTitle').textContent = '首页';
1262 import unittest
1263     <div class="grid">
1264 from fastapi.testclient import TestClient
1265 "hint">资源、场景、交互与发布记录在此汇总展示。</div></div>
1266 from apps.api.main import create_app
1267     }
1268     function renderSimplePage(title, id) {
1269 class HealthEndpointTests(unittest.TestCase):
1270     def test_health_ok(self) -> None:
1271         client = TestClient(create_app())
1272         resp = client.get("/health")
1273         self.assertEqual(resp.status_code, 200)
1274         self.assertEqual(resp.json().get("status"), "ok")
1275     function renderStations() {
1276         document.getElementById('pageTitle').textContent =
1277 if __name__ == "__main__":
1278     unittest.main()
1279     body.innerHTML = `
1280 # apps/api/util.py
1281 # -*- coding: utf-8 -*-
1282 from __future__ import annotations
1283 /div>
1284 import hashlib
1285 import json
1286 from datetime import datetime, timezone
1287 from typing import Any, Dict
1288 from uuid import uuid4
1289     <button onclick="window.__rkModal.openModal('查看',
1290 '查看条目详情与引用关系。')">查看</button>
1291 def new_id() -> str:
1292     return uuid4().hex
1293     </div>
1294     <div class="hint">提示：截图脚本会自动点击以上按钮并截取弹窗状态。
1295 def utc_now() -> str:
1296     return datetime.now(tz=timezone.utc).replace(microsecond=0).isoformat()
1297     }
1298     function route() {
1299 def json_dumps(obj: Any) -> str:
1300     return json.dumps(obj, ensure_ascii=False, separators=(",", ":"))

```

```

1301         if (hash === 'stations') return renderStations();
1302         if (hash === 'zones') return renderSimplePage('资源分区', 'zones');
1303     def sha256_bytes(data: bytes) -> str:
1304         h = hashlib.sha256()
1305         h.update(data)
1306         return h.hexdigest()
1307     'irrigation');
1308         if (hash === 'reports') return renderSimplePage('统计报表', 'reports');
1309     def csv_tags(tags: list[str]) -> str:
1310         clean = []
1311         for t in tags:
1312             t = (t or "").strip()
1313             if not t:
1314                 continue
1315             if "," in t:
1316                 t = t.replace(",", " ")
1317             clean.append(t)
1318         # de-dup, keep order
1319         seen = set()
1320         out = []
1321         for t in clean:
1322             if t in seen:
1323                 continue
1324             seen.add(t)
1325             out.append(t)
1326         return ", ".join(out)
1327     </script>
1328 </body>
1329 def parse_tags(tags_csv: str) -> list[str]:
1330     items = []
1331     for t in (tags_csv or "").split(","):
1332         t = t.strip()
1333         if t:
1334             items.append(t)
1335     return items
1336 # apps/web/admin.html
1337 <!doctype html>
1338 <html lang="zh-CN"><head><meta charset="utf-8" /><meta name="viewport"
1339 content="width=device-width, initial-scale=1" />
1340 <title>管理端 - VR 多模态交互媒介内容创作与发布系统</title></head><body>
1341 <h1>管理端</h1>
1342 <ul>
1343     <li>核心动作按钮口径: 新增 / 编辑 / 查看 / 删除</li>
1344 </ul>
1345 </body></html>
1346 # apps/web/index.html
1347 <!doctype html>
1348 <html lang="zh-CN">
1349     <head>
1350         <meta charset="utf-8" />

```

```

1351     <meta name="viewport" content="width=device-width, initial-scale=1" />
1352     <title>VR 多模态交互媒介内容创作与发布系统</title>
1353 </head>
1354 <body>
1355     <h1>VR 多模态交互媒介内容创作与发布系统</h1>
1356     <ul>
1357         <li><a href="/teacher.html">业务专员端</a></li>
1358         <li><a href="/student.html">员工端</a></li>
1359         <li><a href="/admin.html">管理员端</a></li>
1360     </ul>
1361 </body>
1362 </html>
1363 # apps/web/student.html
1364 <!doctype html>
1365 <html lang="zh-CN"><head><meta charset="utf-8" /><meta name="viewport"
1366 content="width=device-width, initial-scale=1" />
1367 <title>学生端 - VR 多模态交互媒介内容创作与发布系统</title></head><body>
1368 <h1>学生端</h1>
1369 <ul>
1370     <li>当前版本：V1.0（示例）</li>
1371     <li>体验说明：按交互节点顺序体验并记录问题</li>
1372     <li>反馈方式：提交问题编号与截图（后续版本接入反馈单）</li>
1373 </ul>
1374 </body></html>
1375 # apps/web/teacher.html
1376 <!doctype html>
1377 <html lang="zh-CN"><head><meta charset="utf-8" /><meta name="viewpor"
1378 content="width=device-width, initial-scale=1" />
1379 <title>教师端 - VR 多模态交互媒介内容创作与发布系统</title></head><body>
1380 <h1>教师端</h1>
1381 <ol>
1382     <li>查看项目概览与关键指标（资源数、场景数、交互数、最近发布）。</li>
1383     <li>对比两个版本的变更摘要（资源替换、交互规则变更）。</li>
1384     <li>导出发布说明与清单，用于验收留痕。</li>
1385 </ol>
1386 </body></html>
1387 # apps/api/main.py
1388 # -*- coding: utf-8 -*-
1389 from __future__ import annotations
1390 import os
1391 try:
1392     from fastapi import FastAPI
1393     from fastapi.responses import HTMLResponse
1394 except Exception: # pragma: no cover
1395     FastAPI = None # type: ignore[assignment]
1396 from apps.api.routers import (
1397     asset_router,
1398     audit_router,
1399     hr_router,
1400     interaction_router,

```

```

1401     project_router,
1402     scene_router,
1403     versioning_router,
1404 )
1405 def create_app() -> "FastAPI":
1406     if FastAPI is None:
1407         raise RuntimeError("FastAPI is not installed. Install dependencies
1408 before running the API.")
1409     app = FastAPI(title="API")
1410     app.include_router(hr_router)
1411     app.include_router(project_router)
1412     app.include_router(asset_router)
1413     app.include_router(scene_router)
1414     app.include_router(interaction_router)
1415     app.include_router(versioning_router)
1416     app.include_router(audit_router)
1417     @app.get("/", response_class=HTMLResponse)
1418     def index() -> str:
1419         return """<!doctype html>
1420 <html lang="zh-CN">
1421 <head>
1422 <meta charset="utf-8" />
1423 <meta name="viewport" content="width=device-width, initial-scale=1" />
1424 <title>VR 多模态交互媒介内容创作与发布系统</title>
1425 <style>
1426 .root { --bg:#0b1020; --panel:#121a33; --text:#e8ecff; --muted:#aab3d9; -
1427 -accent:#6ee7ff; --danger:#ff5b6e; }
1428 body { margin:0; font-family: "Segoe UI", "Microsoft YaHei", sans-serif;
1429 background:linear-gradient(180deg,#070a14, var(--bg)); color:var(--text); }
1430 header { display:flex; align-items:center; justify-content:space-between;
1431 padding:14px 18px; background:rgba(18,26,51,.8); border-bottom:1px solid
1432 rgba(110,231,255,.18); position:sticky; top:0; backdrop-filter: blur(8px); }
1433 header .brand { font-weight:700; letter-spacing:.5px; }
1434 header .badge { font-size:12px; padding:4px 10px; border:1px solid
1435 rgba(110,231,255,.35); border-radius:999px; color:var(--accent); }
1436 main { max-width:1180px; margin:18px auto; padding:0 18px 28px;
1437 display:grid; grid-template-columns: 260px 1fr; gap:16px; }
1438 nav { background:rgba(18,26,51,.7); border:1px solid rgba(110,231,255,
1439 .12); border-radius:14px; padding:12px; }
1440 nav a { display:block; color:var(--muted); text-decoration:none;
1441 padding:10px 10px; border-radius:10px; }
1442 nav a:hover { background:rgba(110,231,255,.08); color:var(--text); }
1443 .panel { background:rgba(18,26,51,.7); border:1px solid rgba(110,231,255,
1444 .12); border-radius:14px; padding:14px; }
1445 .grid { display:grid; grid-template-columns: repeat(3, 1fr); gap:12px; }
1446 .card { border:1px solid rgba(110,231,255,.12); border-radius:12px;
1447 padding:12px; background:rgba(9,12,24,.55); }
1448 .card h3 { margin:0 0 8px; font-size:14px; color:var(--accent); }
1449 .hint { color:var(--muted); font-size:12px; line-height:1.6; }
1450 .toolbar { display:flex; gap:8px; flex-wrap:wrap; margin:8px 0 6px; }

```

```

1451     button { cursor:pointer; border-radius:10px; border:1px solid rgba(110,
1452 231,255,.22); background:rgba(110,231,255,.10); color:var(--text);
1453 padding:8px 12px; }
1454     button:hover { background:rgba(110,231,255,.16); }
1455     button.danger { border-color:rgba(255,91,110,.35); background:rgba(255,
1456 91,110,.12); }
1457     .k { font-family: ui-monospace, SFMono-Regular, Menlo, Consolas,
1458 monospace; color:var(--accent); }
1459     .hidden { display:none !important; }
1460     #loginBox { max-width:520px; margin:10vh auto; padding:18px; }
1461     #loginBox .row { display:grid; gap:10px; margin-top:10px; }
1462     input { padding:10px 12px; border-radius:10px; border:1px solid rgba(110,
1463 231,255,.18); background:rgba(255,255,255,.05); color:var(--text); }
1464     .row label { color:var(--muted); font-size:12px; }
1465     .row small { color:var(--muted); }
1466     #modal.modal-shell.modal { position:fixed; inset:0; background:rgba(0,0,
1467 0,.55); display:flex; align-items:center; justify-content:center; }
1468     #modal .dialog { width:min(680px, 92vw); border-radius:14px;
1469 background:rgba(18,26,51,.96); border:1px solid rgba(110,231,255,.18);
1470 padding:14px; box-shadow:0 24px 80px rgba(0,0,0,.45); }
1471     #modal .dialog header { position:static; border:none;
1472 background:transparent; padding:0 0 10px; }
1473     #modal .dialog .title { font-weight:700; }
1474     #modal .dialog .content { color:var(--muted); line-height:1.7; font-
1475 size:13px; }
1476 </style>
1477 </head>
1478 <body>
1479 <div id="loginView">
1480 <div id="loginBox" class="panel">
1481 <h1 style="margin:0 0 8px; font-size:18px;
1482 ">VR 多模态交互媒介内容创作与发布系统</h1>
1483 <div class="hint">V1.0 演示环境：用于截图与材料生成的最小可运行界面。<
1484 /div>
1485 <form id="login-form" class="row" onsubmit="return
1486 window.__app.login(event)">
1487 <label>用户名 <input name="username" value="admin" autocomplete=
1488 "username" /></label>
1489 <label>密码 <input name="password" type="password" value="admin123"
1490 autocomplete="current-password" /></label>
1491 <button type="submit">登录</button>
1492 <small>提示：输入任意值均可登录（用于演示与截图）。</small>
1493 </form>
1494 </div>
1495 </div>
1496 <div id="appView" class="hidden">
1497 <header>
1498 <div class="badge">V1.0</div>
1499 </header>
1500 <main>

```

```

1501     <nav>
1502         <a href="/#home">首页</a>
1503         <a href="/#zones">资源分区</a>
1504         <a href="/#stations">工作台（Stations）</a>
1505         <a href="/#inspections">交互检查</a>
1506         <a href="/#alerts">风险提醒</a>
1507         <a href="/#irrigation">发布流水</a>
1508         <a href="/#reports">统计报表</a>
1509     </nav>
1510     <section class="panel">
1511         <div id="pageTitle" style="font-weight:700; margin-bottom:10px;
1512 ">首页</div>
1513         <div id="pageBody"></div>
1514     </section>
1515 </main>
1516 </div>
1517 <div id="modal" class="modal-shell modal hidden">
1518     <div class="dialog" role="dialog" aria-modal="true">
1519         <header>
1520             <div class="title" id="modalTitle">操作</div>
1521             <button class="danger" onclick="window.__rkModal.closeModal()
1522 ">关闭</button>
1523         </header>
1524         <div class="content" id="modalContent"></div>
1525     </div>
1526 </div>
1527 <script>
1528     window.__rkModal = {
1529         closeModal() {
1530             document.getElementById('modal').classList.add('hidden');
1531         },
1532         openModal(title, content) {
1533             document.getElementById('modalTitle').textContent = title;
1534             document.getElementById('modalContent').textContent = content;
1535             document.getElementById('modal').classList.remove('hidden');
1536         }
1537     };
1538     function renderHome() {
1539         document.getElementById('pageTitle').textContent = '首页';
1540         document.getElementById('pageBody').innerHTML = `
1541             <div class="grid">
1542                 <div class="card"><h3>项目概览</h3><div class=
1543 "hint">资源、场景、交互与发布记录在此汇总展示。</div></div>
1544             </div>`;
1545     }
1546     function renderSimplePage(title, id) {
1547         document.getElementById('pageTitle').textContent = title;
1548         const body = document.getElementById('pageBody');
1549         body.innerHTML = `<div id="${id}" class="card">
1550             <h3>${title}</h3>

```

```

1551     </div>`;
1552 }
1553 function renderStations() {
1554     document.getElementById('pageTitle').textContent =
1555 '工作台（Stations）';
1556     const body = document.getElementById('pageBody');
1557     body.innerHTML = `
1558     <div id="stations" class="card">
1559         <h3>Stations 列表</h3>
1560         <div class="hint">用于演示 “新增/编辑/查看/删除” 弹窗操作按钮文案。<
1561 /div>
1562         <div class="toolbar">
1563             <button onclick="window.__rkModal.openModal('新增',
1564 '创建新的条目并填写字段。')">新增</button>
1565             <button onclick="window.__rkModal.openModal('编辑',
1566 '修改条目名称、标签与说明。')">编辑</button>
1567             <button onclick="window.__rkModal.openModal('查看',
1568 '查看条目详情与引用关系。')">查看</button>
1569             <button class="danger" onclick="window.__rkModal.openModal('删除
1570 ', '删除条目前请确认无引用并记录审计。')">删除</button>
1571         </div>
1572         <div class="hint">提示：截图脚本会自动点击以上按钮并截取弹窗状态。
1573 </div>
1574     </div>`;
1575 }
1576 function route() {
1577     const hash = (location.hash || '#home').replace(/^#/, "");
1578     if (hash === 'home' || hash === "") return renderHome();
1579     if (hash === 'stations') return renderStations();
1580     if (hash === 'zones') return renderSimplePage('资源分区', 'zones');
1581     if (hash === 'inspections') return renderSimplePage('交互检查',
1582 'inspections');
1583     if (hash === 'alerts') return renderSimplePage('风险提醒', 'alerts');
1584     if (hash === 'irrigation') return renderSimplePage('发布流水',
1585 'irrigation');
1586     if (hash === 'reports') return renderSimplePage('统计报表', 'reports');
1587     return renderSimplePage('页面', hash);
1588 }
1589 window.__app = {
1590     login(e) {
1591         e.preventDefault();
1592         localStorage.setItem('__demo_logged_in', '1');
1593         document.getElementById('loginView').classList.add('hidden');
1594         document.getElementById('appView').classList.remove('hidden');
1595         route();
1596         return false;
1597     }
1598 };
1599 window.addEventListener('hashchange', () => route());
1600 if (localStorage.getItem('__demo_logged_in') === '1') {

```



```

1601     document.getElementById('loginView').classList.add('hidden');
1602     document.getElementById('appView').classList.remove('hidden');
1603     route();
1604 }
1605 </script>
1606 </body>
1607 </html>>"""
1608 @app.get("/health")
1609 def health() -> dict[str, str]:
1610     return {"status": "ok"}
1611 return app
1612 app = create_app()
1613 def _main() -> None:
1614     import uvicorn
1615     port = int(os.environ.get("APP_PORT", "8010"))
1616     uvicorn.run("apps.api.main:app", host="127.0.0.1", port=port, log_level=
1617 "warning")
1618 if __name__ == "__main__":
1619     _main()
1620 # apps/api/routers/_init_.py
1621 # -*- coding: utf-8 -*-
1622 from __future__ import annotations
1623 from .hr import router as hr_router
1624 from .project import router as project_router
1625 from .asset import router as asset_router
1626 from .scene import router as scene_router
1627 from .interaction import router as interaction_router
1628 from .versioning import router as versioning_router
1629 from .audit import router as audit_router
1630 __all__ = [
1631     "hr_router",
1632     "project_router",
1633     "asset_router",
1634     "scene_router",
1635     "interaction_router",
1636     "versioning_router",
1637     "audit_router",
1638 ]
1639 # apps/api/routers/asset.py
1640 # -*- coding: utf-8 -*-
1641 from __future__ import annotations
1642 from fastapi import APIRouter, HTTPException
1643 from apps.api.schemas.asset import AssetCreate, AssetOut
1644 from apps.api.services import asset_repo
1645 router = APIRouter(prefix="/api/assets", tags=["assets"])
1646 @router.get("", response_model=list[AssetOut])
1647 def list_assets(project_id: str):
1648     return asset_repo.list_assets(project_id)
1649 @router.post("", response_model=dict)
1650 def create_asset(payload: AssetCreate):

```

```

1651     aid = asset_repo.create_asset(
1652         payload.project_id,
1653         payload.type,
1654         payload.filename,
1655         payload.sha256,
1656         payload.size,
1657         payload.tags,
1658         payload.meta_json,
1659     )
1660     return {"id": aid}
1661 @router.delete("/{asset_id}", response_model=dict)
1662 def delete_asset(asset_id: str):
1663     ok = asset_repo.delete_asset(asset_id)
1664     if not ok:
1665         raise HTTPException(status_code=404, detail="not found")
1666     return {"deleted": True}
1667 # apps/api/routers/audit.py
1668 # -*- coding: utf-8 -*-
1669 from __future__ import annotations
1670 from fastapi import APIRouter
1671 from apps.api.services import audit_repo
1672 router = APIRouter(prefix="/api/audit", tags=["audit"])
1673 @router.get("", response_model=list[dict])
1674 def list_audit(limit: int = 100):
1675     return audit_repo.list_audit(limit=limit)
1676 # apps/api/routers/hr.py
1677 # -*- coding: utf-8 -*-
1678 from __future__ import annotations
1679 from fastapi import APIRouter, HTTPException
1680 from pydantic import BaseModel, Field
1681 from apps.api.services.store import InMemoryStore
1682 router = APIRouter(prefix="/api/hr", tags=["hr"])
1683 class DepartmentIn(BaseModel):
1684     name: str = Field(..., min_length=2, max_length=50)
1685 class PositionIn(BaseModel):
1686     name: str = Field(..., min_length=2, max_length=50)
1687     dept_id: str
1688 class EmployeeIn(BaseModel):
1689     name: str = Field(..., min_length=2, max_length=50)
1690     dept_id: str
1691     title: str = Field(..., min_length=1, max_length=50)
1692 def get_store() -> InMemoryStore:
1693     # module-level singleton to keep the example minimal.
1694     global _STORE
1695     try:
1696         return _STORE
1697     except NameError:
1698         _STORE = InMemoryStore()
1699     return _STORE
1700 @router.get("/departments")

```

```

1701 def list_departments() -> list[dict]:
1702     return get_store().list_departments()
1703 @router.post("/departments")
1704 def create_department(payload: DepartmentIn) -> dict:
1705     _id = get_store().create_department(payload.name)
1706     return {"id": _id}
1707 @router.delete("/departments/{dept_id}")
1708 def delete_department(dept_id: str) -> dict:
1709     ok = get_store().delete_department(dept_id)
1710     if not ok:
1711         raise HTTPException(status_code=404, detail="not found")
1712     return {"deleted": True}
1713 @router.get("/positions")
1714 def list_positions() -> list[dict]:
1715     return get_store().list_positions()
1716 @router.post("/positions")
1717 def create_position(payload: PositionIn) -> dict:
1718     _id = get_store().create_position(payload.name, payload.dept_id)
1719     return {"id": _id}
1720 @router.delete("/positions/{pos_id}")
1721 def delete_position(pos_id: str) -> dict:
1722     ok = get_store().delete_position(pos_id)
1723     if not ok:
1724         raise HTTPException(status_code=404, detail="not found")
1725     return {"deleted": True}
1726 @router.get("/employees")
1727 def list_employees() -> list[dict]:
1728     return get_store().list_employees()
1729 @router.post("/employees")
1730 def create_employee(payload: EmployeeIn) -> dict:
1731     _id = get_store().create_employee(payload.name, payload.dept_id,
1732     payload.title)
1733     return {"id": _id}
1734 @router.delete("/employees/{emp_id}")
1735 def delete_employee(emp_id: str) -> dict:
1736     ok = get_store().delete_employee(emp_id)
1737     if not ok:
1738         raise HTTPException(status_code=404, detail="not found")
1739     return {"deleted": True}
1740 # apps/api/routers/interaction.py
1741 # -*- coding: utf-8 -*-
1742 from __future__ import annotations
1743 from fastapi import APIRouter
1744 from apps.api.schemas.interaction import InteractionCreate, InteractionOut
1745 from apps.api.services import interaction_repo
1746 router = APIRouter(prefix="/api/interactions", tags=["interactions"])
1747 @router.post("", response_model=dict)
1748 def create_interaction(payload: InteractionCreate):
1749     iid = interaction_repo.create_interaction(payload.project_id,
1750     payload.scene_id, payload.name, payload.enabled, payload.graph_json)

```

```

1751     return {"id": iid}
1752 @router.get("", response_model=list[InteractionOut])
1753 def list_interactions(project_id: str):
1754     return interaction_repo.list_interactions(project_id)
1755 # apps/api/routers/project.py
1756 # -*- coding: utf-8 -*-
1757 from __future__ import annotations
1758 from fastapi import APIRouter, HTTPException
1759 from apps.api.schemas.project import ProjectCreate, ProjectOut,
1760 ProjectUpdate
1761 from apps.api.services import project_repo
1762 router = APIRouter(prefix="/api/projects", tags=["projects"])
1763 @router.get("", response_model=list[ProjectOut])
1764 def list_projects():
1765     return project_repo.list_projects()
1766 @router.post("", response_model=dict)
1767 def create_project(payload: ProjectCreate):
1768     pid = project_repo.create_project(payload.name, payload.target_device,
1769 payload.constraints_json)
1770     return {"id": pid}
1771 @router.get("/{project_id}", response_model=ProjectOut)
1772 def get_project(project_id: str):
1773     p = project_repo.get_project(project_id)
1774     if not p:
1775         raise HTTPException(status_code=404, detail="not found")
1776     return p
1777 @router.patch("/{project_id}", response_model=dict)
1778 def patch_project(project_id: str, payload: ProjectUpdate):
1779     ok = project_repo.update_project(project_id, payload.model_dump())
1780     if not ok:
1781         raise HTTPException(status_code=404, detail="not found or no
1782 changes")
1783     return {"updated": True}
1784 @router.delete("/{project_id}", response_model=dict)
1785 def delete_project(project_id: str):
1786     ok = project_repo.delete_project(project_id)
1787     if not ok:
1788         raise HTTPException(status_code=404, detail="not found")
1789     return {"deleted": True}
1790 # apps/api/routers/publish.py
1791 # apps/api/routers/scene.py
1792 # -*- coding: utf-8 -*-
1793 from __future__ import annotations
1794 from fastapi import APIRouter, HTTPException
1795 from apps.api.schemas.scene import SceneCreate, SceneNodeCreate,
1796 SceneNodeOut, SceneOut
1797 from apps.api.services import scene_repo
1798 router = APIRouter(prefix="/api/scenes", tags=["scenes"])
1799 @router.post("", response_model=dict)
1800 def create_scene(payload: SceneCreate):

```

```

1801     sid = scene_repo.create_scene(payload.project_id, payload.name)
1802     return {"id": sid}
1803 @router.get("", response_model=list[SceneOut])
1804 def list_scenes(project_id: str):
1805     return scene_repo.list_scenes(project_id)
1806 @router.post("/nodes", response_model=dict)
1807 def create_node(payload: SceneNodeCreate):
1808     nid = scene_repo.create_node(
1809         payload.scene_id,
1810         payload.parent_id,
1811         payload.name,
1812         payload.node_type,
1813         payload.transform_json,
1814         payload.asset_ref_id,
1815         payload.props_json,
1816     )
1817     return {"id": nid}
1818 @router.get("/{scene_id}/nodes", response_model=list[SceneNodeOut])
1819 def list_nodes(scene_id: str):
1820     return scene_repo.list_nodes(scene_id)
1821 # apps/api/routers/versioning.py
1822 # -*- coding: utf-8 -*-
1823 from __future__ import annotations
1824 from fastapi import APIRouter
1825 from apps.api.schemas.versioning import PublishCreate, PublishOut,
1826 VersionCreate, VersionOut
1827 from apps.api.services import publish_repo, versioning_repo
1828 router = APIRouter(prefix="/api/versioning", tags=["versioning"])
1829 @router.post("/versions", response_model=dict)
1830 def create_version(payload: VersionCreate):
1831     vid = versioning_repo.create_version(payload.project_id, payload.version,
1832     payload.note)
1833     return {"id": vid}
1834 @router.get("/versions", response_model=list[VersionOut])
1835 def list_versions(project_id: str):
1836     return versioning_repo.list_versions(project_id)
1837 @router.post("/publish", response_model=dict)
1838 def create_publish(payload: PublishCreate):
1839     pid = publish_repo.create_publish_record(payload.project_id,
1840     payload.version_id, payload.channel, payload.note, payload.artifact_path)
1841     return {"id": pid}
1842 @router.get("/publish", response_model=list[PublishOut])
1843 def list_publish(project_id: str):
1844     return publish_repo.list_publish_records(project_id)
1845 # apps/api/domain/_init_.py
1846 # apps/api/domain/models.py
1847 # -*- coding: utf-8 -*-
1848 from __future__ import annotations
1849 from dataclasses import dataclass
1850 from datetime import datetime

```

```

1851 @dataclass(frozen=True)
1852 class Department:
1853     id: str
1854     name: str
1855     created_at: datetime
1856 @dataclass(frozen=True)
1857 class Position:
1858     id: str
1859     name: str
1860     dept_id: str
1861     created_at: datetime
1862 @dataclass(frozen=True)
1863 class Employee:
1864     id: str
1865     name: str
1866     dept_id: str
1867     title: str
1868     created_at: datetime
1869 # apps/api/services/__init__.py
1870 # apps/api/services/asset_repo.py
1871 # -*- coding: utf-8 -*-
1872 from __future__ import annotations
1873 import sqlite3
1874 from apps.api.db import db
1875 from apps.api.util import csv_tags, new_id, parse_tags, utc_now
1876 def create_asset(project_id: str, type: str, filename: str, sha256: str,
1877 size: int, tags: list[str], meta_json: str) -> str:
1878     aid = new_id()
1879     with db() as conn:
1880         conn.execute(
1881             "INSERT INTO asset(id,project_id,type,filename,sha256,size,tags,
1882 meta_json,created_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
1883             (aid, project_id, type, filename, sha256, int(size),
1884 csv_tags(tags), meta_json or "{}", utc_now()),
1885         )
1886     return aid
1887 def list_assets(project_id: str) -> list[dict]:
1888     with db() as conn:
1889         rows = conn.execute("SELECT * FROM asset WHERE project_id=? ORDER BY
1890 created_at DESC", (project_id,)).fetchall()
1891         out = []
1892         for r in rows:
1893             d = dict(r)
1894             d["tags"] = parse_tags(d.get("tags", ""))
1895             out.append(d)
1896     return out
1897 def delete_asset(asset_id: str) -> bool:
1898     with db() as conn:
1899         cur = conn.execute("DELETE FROM asset WHERE id=?", (asset_id,))
1900         return cur.rowcount > 0

```

```

1901 # apps/api/services/audit_repo.py
1902 # -*- coding: utf-8 -*-
1903 from __future__ import annotations
1904 from apps.api.db import db
1905 from apps.api.util import json_dumps, new_id, utc_now
1906 def write_audit(actor: str, action: str, target_type: str, target_id: str,
1907 detail: dict) -> str:
1908     aid = new_id()
1909     with db() as conn:
1910         conn.execute(
1911             "INSERT INTO audit_log(id,actor,action,target_type,target_id,
1912 detail_json,created_at) VALUES (?, ?, ?, ?, ?, ?, ?)",
1913             (aid, actor or "system", action, target_type, target_id,
1914 json_dumps(detail or {}), utc_now()),
1915         )
1916     return aid
1917 def list_audit(limit: int = 100) -> list[dict]:
1918     with db() as conn:
1919         rows = conn.execute("SELECT * FROM audit_log ORDER BY created_at
1920 DESC LIMIT ?", (int(limit),)).fetchall()
1921         return [dict(r) for r in rows]
1922 # apps/api/services/interaction_repo.py
1923 # -*- coding: utf-8 -*-
1924 from __future__ import annotations
1925 from apps.api.db import db
1926 from apps.api.util import new_id, utc_now
1927 def create_interaction(project_id: str, scene_id: str, name: str, enabled:
1928 bool, graph_json: str) -> str:
1929     iid = new_id()
1930     with db() as conn:
1931         conn.execute(
1932             "INSERT INTO interaction(id,project_id,scene_id,name,enabled,
1933 graph_json,created_at) VALUES (?, ?, ?, ?, ?, ?, ?)",
1934             (iid, project_id, scene_id, name, 1 if enabled else 0,
1935 graph_json or "{}", utc_now()),
1936         )
1937     return iid
1938 def list_interactions(project_id: str) -> list[dict]:
1939     with db() as conn:
1940         rows = conn.execute("SELECT * FROM interaction WHERE project_id=?
1941 ORDER BY created_at DESC", (project_id,)).fetchall()
1942         out = []
1943         for r in rows:
1944             d = dict(r)
1945             d["enabled"] = bool(d.get("enabled"))
1946             out.append(d)
1947     return out
1948 # apps/api/services/project_repo.py
1949 # -*- coding: utf-8 -*-
1950 from __future__ import annotations

```

```

1951 import sqlite3
1952 from apps.api.db import db
1953 from apps.api.util import json_dumps, new_id, utc_now
1954 def create_project(name: str, target_device: str, constraints_json: str) ->
1955 str:
1956     pid = new_id()
1957     with db() as conn:
1958         conn.execute(
1959             "INSERT INTO project(id,name,target_device,constraints_json,
1960 status,created_at) VALUES (?, ?, ?, ?, ?, ?)",
1961             (pid, name, target_device, constraints_json or "{}", "draft",
1962             utc_now()),
1963             )
1964     return pid
1965 def list_projects() -> list[dict]:
1966     with db() as conn:
1967         rows = conn.execute("SELECT * FROM project ORDER BY created_at DESC")
1968     .fetchall()
1969     return [dict(r) for r in rows]
1970 def get_project(project_id: str) -> dict | None:
1971     with db() as conn:
1972         row = conn.execute("SELECT * FROM project WHERE id=?", (project_id,))
1973     .fetchone()
1974     return dict(row) if row else None
1975 def update_project(project_id: str, patch: dict) -> bool:
1976     allow = {"name", "target_device", "constraints_json", "status"}
1977     fields = {k: v for k, v in patch.items() if k in allow and v is not None}
1978     if not fields:
1979         return False
1980     sets = ", ".join([f"{k}=?" for k in fields.keys()])
1981     values = list(fields.values()) + [project_id]
1982     with db() as conn:
1983         cur = conn.execute(f"UPDATE project SET {sets} WHERE id=?", values)
1984         return cur.rowcount > 0
1985 def delete_project(project_id: str) -> bool:
1986     with db() as conn:
1987         cur = conn.execute("DELETE FROM project WHERE id=?", (project_id,))
1988         return cur.rowcount > 0
1989 # apps/api/services/publish_repo.py
1990 # -*- coding: utf-8 -*-
1991 from __future__ import annotations
1992 from apps.api.db import db
1993 from apps.api.util import new_id, utc_now
1994 def create_publish_record(project_id: str, version_id: str, channel: str,
1995 note: str, artifact_path: str) -> str:
1996     pid = new_id()
1997     with db() as conn:
1998         conn.execute(
1999             "INSERT INTO publish_record(id,project_id,version_id,channel,
2000 note,artifact_path,created_at) VALUES (?, ?, ?, ?, ?, ?, ?)",

```



```

2001         (pid, project_id, version_id, channel or "local_export", note or
2002         "", artifact_path or "", utc_now()),
2003     )
2004     return pid
2005 def list_publish_records(project_id: str) -> list[dict]:
2006     with db() as conn:
2007         rows = conn.execute("SELECT * FROM publish_record WHERE project_id=?
2008 ORDER BY created_at DESC", (project_id,)).fetchall()
2009         return [dict(r) for r in rows]
2010 # apps/api/services/scene_repo.py
2011 # -*- coding: utf-8 -*-
2012 from __future__ import annotations
2013 from apps.api.db import db
2014 from apps.api.util import new_id, utc_now
2015 def create_scene(project_id: str, name: str) -> str:
2016     sid = new_id()
2017     with db() as conn:
2018         conn.execute("INSERT INTO scene(id,project_id,name,created_at)
2019 VALUES (?,?,,?)", (sid, project_id, name, utc_now()))
2020     return sid
2021 def list_scenes(project_id: str) -> list[dict]:
2022     with db() as conn:
2023         rows = conn.execute("SELECT * FROM scene WHERE project_id=? ORDER BY
2024 created_at DESC", (project_id,)).fetchall()
2025         return [dict(r) for r in rows]
2026 def create_node(scene_id: str, parent_id: str | None, name: str, node_type:
2027 str, transform_json: str, asset_ref_id: str | None, props_json: str) -> str:
2028     nid = new_id()
2029     with db() as conn:
2030         conn.execute(
2031             "INSERT INTO scene_node(id,scene_id,parent_id,name,node_type,
2032 transform_json,asset_ref_id,props_json,created_at) VALUES (?,?,,?,?,,?)
2033 ",
2034             (nid, scene_id, parent_id, name, node_type, transform_json or "{}
2035 ", asset_ref_id, props_json or "{}", utc_now()),
2036         )
2037     return nid
2038 def list_nodes(scene_id: str) -> list[dict]:
2039     with db() as conn:
2040         rows = conn.execute("SELECT * FROM scene_node WHERE scene_id=? ORDER
2041 BY created_at ASC", (scene_id,)).fetchall()
2042         return [dict(r) for r in rows]
2043 # apps/api/services/store.py
2044 # -*- coding: utf-8 -*-
2045 from __future__ import annotations
2046 from dataclasses import asdict
2047 from datetime import datetime
2048 from typing import Any
2049 from uuid import uuid4
2050 class InMemoryStore:

```

```

2051     def __init__(self) -> None:
2052         self._departments: dict[str, dict[str, Any]] = {}
2053         self._positions: dict[str, dict[str, Any]] = {}
2054         self._employees: dict[str, dict[str, Any]] = {}
2055     def _new_id(self) -> str:
2056         return uuid4().hex
2057     def list_departments(self) -> list[dict[str, Any]]:
2058         return list(self._departments.values())
2059     def create_department(self, name: str) -> str:
2060         _id = self._new_id()
2061         self._departments[_id] = {"id": _id, "name": name, "created_at":
2062     datetime.utcnow().isoformat()}
2063         return _id
2064     def delete_department(self, dept_id: str) -> bool:
2065         return self._departments.pop(dept_id, None) is not None
2066     def list_positions(self) -> list[dict[str, Any]]:
2067         return list(self._positions.values())
2068     def create_position(self, name: str, dept_id: str) -> str:
2069         _id = self._new_id()
2070         self._positions[_id] = {"id": _id, "name": name, "dept_id": dept_id,
2071     "created_at": datetime.utcnow().isoformat()}
2072         return _id
2073     def delete_position(self, pos_id: str) -> bool:
2074         return self._positions.pop(pos_id, None) is not None
2075     def list_employees(self) -> list[dict[str, Any]]:
2076         return list(self._employees.values())
2077     def create_employee(self, name: str, dept_id: str, title: str) -> str:
2078         _id = self._new_id()
2079         self._employees[_id] = {
2080             "id": _id,
2081             "name": name,
2082             "dept_id": dept_id,
2083             "title": title,
2084             "created_at": datetime.utcnow().isoformat(),
2085         }
2086         return _id
2087     def delete_employee(self, emp_id: str) -> bool:
2088         return self._employees.pop(emp_id, None) is not None
2089 # apps/api/services/versioning_repo.py
2090 # -*- coding: utf-8 -*-
2091 from __future__ import annotations
2092 import json
2093 from apps.api.db import db
2094 from apps.api.util import json_dumps, new_id, utc_now
2095 def _asset_manifest(conn, project_id: str) -> list[dict]:
2096     rows = conn.execute(
2097         "SELECT id,type,filename,sha256,size,tags,meta_json,created_at FROM
2098     asset WHERE project_id=? ORDER BY created_at ASC",
2099         (project_id,),
2100     ).fetchall()

```

```

2101         return [dict(r) for r in rows]
2102 def _snapshot(conn, project_id: str) -> dict:
2103     scenes = conn.execute("SELECT id,name,created_at FROM scene WHERE
2104 project_id=? ORDER BY created_at ASC", (project_id,)).fetchall()
2105     interactions = conn.execute(
2106         "SELECT id,scene_id,name,enabled,graph_json,created_at FROM
2107 interaction WHERE project_id=? ORDER BY created_at ASC",
2108         (project_id,),
2109     ).fetchall()
2110     return {
2111         "scenes": [dict(r) for r in scenes],
2112         "interactions": [dict(r) for r in interactions],
2113     }
2114 def create_version(project_id: str, version: str, note: str) -> str:
2115     vid = new_id()
2116     with db() as conn:
2117         manifest = _asset_manifest(conn, project_id)
2118         snap = _snapshot(conn, project_id)
2119         conn.execute(
2120             "INSERT INTO version(id,project_id,version,note,snapshot_json,
2121 asset_manifest_json,created_at) VALUES (?,?,,?,,?,,?)",
2122             (vid, project_id, version, note or "", json_dumps(snap),
2123 json_dumps(manifest), utc_now()),
2124         )
2125     return vid
2126 def list_versions(project_id: str) -> list[dict]:
2127     with db() as conn:
2128         rows = conn.execute("SELECT * FROM version WHERE project_id=? ORDER
2129 BY created_at DESC", (project_id,)).fetchall()
2130     return [dict(r) for r in rows]
2131 # apps/api/schemas/common.py
2132 # -*- coding: utf-8 -*-
2133 from __future__ import annotations
2134 from pydantic import BaseModel, Field
2135 class IdResponse(BaseModel):
2136     id: str = Field(..., description="资源唯一标识")
2137 # apps/__init__.py
2138 # apps/api/__init__.py
2139 # apps/api/db.py
2140 # -*- coding: utf-8 -*-
2141 from __future__ import annotations
2142 import os
2143 import sqlite3
2144 from contextlib import contextmanager
2145 from pathlib import Path
2146 from typing import Iterator
2147 def resolve_db_path() -> Path:
2148     base = os.environ.get("APP_DB_DIR", "").strip()
2149     root = Path(base).resolve() if base else Path.cwd().resolve()
2150     return root / "data" / "app.db"

```

```

2151 def ensure_db_schema(conn: sqlite3.Connection) -> None:
2152     conn.execute("PRAGMA foreign_keys = ON;")
2153     conn.executescript(
2154         """
2155 CREATE TABLE IF NOT EXISTS project (
2156     id TEXT PRIMARY KEY,
2157     name TEXT NOT NULL,
2158     target_device TEXT NOT NULL,
2159     constraints_json TEXT NOT NULL,
2160     status TEXT NOT NULL,
2161     created_at TEXT NOT NULL
2162 );
2163 CREATE TABLE IF NOT EXISTS asset (
2164     id TEXT PRIMARY KEY,
2165     project_id TEXT NOT NULL,
2166     type TEXT NOT NULL,
2167     filename TEXT NOT NULL,
2168     sha256 TEXT NOT NULL,
2169     size INTEGER NOT NULL,
2170     tags TEXT NOT NULL,
2171     meta_json TEXT NOT NULL,
2172     created_at TEXT NOT NULL,
2173     FOREIGN KEY(project_id) REFERENCES project(id) ON DELETE CASCADE
2174 );
2175 CREATE INDEX IF NOT EXISTS idx_asset_project ON asset(project_id);
2176 CREATE UNIQUE INDEX IF NOT EXISTS idx_asset_unique_hash ON asset(project_id,
2177 sha256);
2178 CREATE TABLE IF NOT EXISTS scene (
2179     id TEXT PRIMARY KEY,
2180     project_id TEXT NOT NULL,
2181     name TEXT NOT NULL,
2182     created_at TEXT NOT NULL,
2183     FOREIGN KEY(project_id) REFERENCES project(id) ON DELETE CASCADE
2184 );
2185 CREATE INDEX IF NOT EXISTS idx_scene_project ON scene(project_id);
2186 CREATE TABLE IF NOT EXISTS scene_node (
2187     id TEXT PRIMARY KEY,
2188     scene_id TEXT NOT NULL,
2189     parent_id TEXT,
2190     name TEXT NOT NULL,
2191     node_type TEXT NOT NULL,
2192     transform_json TEXT NOT NULL,
2193     asset_ref_id TEXT,
2194     props_json TEXT NOT NULL,
2195     created_at TEXT NOT NULL,
2196     FOREIGN KEY(scene_id) REFERENCES scene(id) ON DELETE CASCADE
2197 );
2198 CREATE INDEX IF NOT EXISTS idx_node_scene ON scene_node(scene_id);
2199 CREATE TABLE IF NOT EXISTS interaction (
2200     id TEXT PRIMARY KEY,

```

```

2201     project_id TEXT NOT NULL,
2202     scene_id TEXT NOT NULL,
2203     name TEXT NOT NULL,
2204     enabled INTEGER NOT NULL,
2205     graph_json TEXT NOT NULL,
2206     created_at TEXT NOT NULL,
2207     FOREIGN KEY(project_id) REFERENCES project(id) ON DELETE CASCADE,
2208     FOREIGN KEY(scene_id) REFERENCES scene(id) ON DELETE CASCADE
2209 );
2210 CREATE INDEX IF NOT EXISTS idx_interaction_project ON interaction(project_id)
2211 ;
2212 CREATE TABLE IF NOT EXISTS version (
2213     id TEXT PRIMARY KEY,
2214     project_id TEXT NOT NULL,
2215     version TEXT NOT NULL,
2216     note TEXT NOT NULL,
2217     snapshot_json TEXT NOT NULL,
2218     asset_manifest_json TEXT NOT NULL,
2219     created_at TEXT NOT NULL,
2220     FOREIGN KEY(project_id) REFERENCES project(id) ON DELETE CASCADE
2221 );
2222 CREATE UNIQUE INDEX IF NOT EXISTS idx_version_unique ON version(project_id,
2223 version);
2224 CREATE TABLE IF NOT EXISTS publish_record (
2225     id TEXT PRIMARY KEY,
2226     project_id TEXT NOT NULL,
2227     version_id TEXT NOT NULL,
2228     channel TEXT NOT NULL,
2229     note TEXT NOT NULL,
2230     artifact_path TEXT NOT NULL,
2231     created_at TEXT NOT NULL,
2232     FOREIGN KEY(project_id) REFERENCES project(id) ON DELETE CASCADE,
2233     FOREIGN KEY(version_id) REFERENCES version(id) ON DELETE CASCADE
2234 );
2235 CREATE INDEX IF NOT EXISTS idx_publish_project ON publish_record(project_id);
2236 CREATE TABLE IF NOT EXISTS audit_log (
2237     id TEXT PRIMARY KEY,
2238     actor TEXT NOT NULL,
2239     action TEXT NOT NULL,
2240     target_type TEXT NOT NULL,
2241     target_id TEXT NOT NULL,
2242     detail_json TEXT NOT NULL,
2243     created_at TEXT NOT NULL
2244 );
2245 CREATE INDEX IF NOT EXISTS idx_audit_ts ON audit_log(created_at);
2246 """
2247 )
2248 @contextmanager
2249 def db()-> Iterator[sqlite3.Connection]:
2250     path = resolve_db_path()

```

```

2251     path.parent.mkdir(parents=True, exist_ok=True)
2252     conn = sqlite3.connect(str(path))
2253     conn.row_factory = sqlite3.Row
2254     ensure_db_schema(conn)
2255     try:
2256         yield conn
2257         conn.commit()
2258     finally:
2259         conn.close()
2260 # apps/api/schemas/__init__.py
2261 # apps/api/schemas/asset.py
2262 # -*- coding: utf-8 -*-
2263 from __future__ import annotations
2264 from pydantic import BaseModel, Field
2265 class AssetCreate(BaseModel):
2266     project_id: str = Field(..., min_length=8, max_length=64)
2267     type: str = Field(..., min_length=2, max_length=20)
2268     filename: str = Field(..., min_length=1, max_length=160)
2269     sha256: str = Field(..., min_length=16, max_length=80)
2270     size: int = Field(..., ge=0, le=2_000_000_000)
2271     tags: list[str] = Field(default_factory=list)
2272     meta_json: str = Field("{}", max_length=8000)
2273 class AssetOut(BaseModel):
2274     id: str
2275     project_id: str
2276     type: str
2277     filename: str
2278     sha256: str
2279     size: int
2280     tags: list[str]
2281     meta_json: str
2282     created_at: str
2283 # apps/api/schemas/interaction.py
2284 # -*- coding: utf-8 -*-
2285 from __future__ import annotations
2286 from pydantic import BaseModel, Field
2287 class InteractionCreate(BaseModel):
2288     project_id: str = Field(..., min_length=8, max_length=64)
2289     scene_id: str = Field(..., min_length=8, max_length=64)
2290     name: str = Field(..., min_length=2, max_length=80)
2291     enabled: bool = Field(True)
2292     graph_json: str = Field("{}", max_length=20000)
2293 class InteractionOut(BaseModel):
2294     id: str
2295     project_id: str
2296     scene_id: str
2297     name: str
2298     enabled: bool
2299     graph_json: str
2300     created_at: str

```

```

2301 # apps/api/schemas/project.py
2302 # -*- coding: utf-8 -*-
2303 from __future__ import annotations
2304 from pydantic import BaseModel, Field
2305 class ProjectCreate(BaseModel):
2306     name: str = Field(..., min_length=2, max_length=80, description=
2307 "项目名称")
2308     target_device: str = Field(..., min_length=2, max_length=30, description=
2309 "目标设备")
2310     constraints_json: str = Field({}, max_length=4000, description=
2311 "性能/规格约束 JSON")
2312 class ProjectUpdate(BaseModel):
2313     name: str | None = Field(None, min_length=2, max_length=80)
2314     target_device: str | None = Field(None, min_length=2, max_length=30)
2315     constraints_json: str | None = Field(None, max_length=4000)
2316     status: str | None = Field(None, max_length=30)
2317 class ProjectOut(BaseModel):
2318     id: str
2319     name: str
2320     target_device: str
2321     constraints_json: str
2322     status: str
2323     created_at: str
2324 # apps/api/schemas/scene.py
2325 # -*- coding: utf-8 -*-
2326 from __future__ import annotations
2327 from pydantic import BaseModel, Field
2328 class SceneCreate(BaseModel):
2329     project_id: str = Field(..., min_length=8, max_length=64)
2330     name: str = Field(..., min_length=2, max_length=80)
2331 class SceneOut(BaseModel):
2332     id: str
2333     project_id: str
2334     name: str
2335     created_at: str
2336 class SceneNodeCreate(BaseModel):
2337     scene_id: str = Field(..., min_length=8, max_length=64)
2338     parent_id: str | None = Field(None, max_length=64)
2339     name: str = Field(..., min_length=1, max_length=80)
2340     node_type: str = Field(..., min_length=2, max_length=30)
2341     transform_json: str = Field({}, max_length=4000)
2342     asset_ref_id: str | None = Field(None, max_length=64)
2343     props_json: str = Field({}, max_length=8000)
2344 class SceneNodeOut(BaseModel):
2345     id: str
2346     scene_id: str
2347     parent_id: str | None
2348     name: str
2349     node_type: str
2350     transform_json: str

```

```

2351     asset_ref_id: str | None
2352     props_json: str
2353     created_at: str
2354 # apps/api/schemas/versioning.py
2355 # -*- coding: utf-8 -*-
2356 from __future__ import annotations
2357 from pydantic import BaseModel, Field
2358 class VersionCreate(BaseModel):
2359     project_id: str = Field(..., min_length=8, max_length=64)
2360     version: str = Field(..., min_length=2, max_length=20, description=
2361 "版本号，如 V1.0.0")
2362     note: str = Field("", max_length=1000)
2363 class VersionOut(BaseModel):
2364     id: str
2365     project_id: str
2366     version: str
2367     note: str
2368     snapshot_json: str
2369     asset_manifest_json: str
2370     created_at: str
2371 class PublishCreate(BaseModel):
2372     project_id: str = Field(..., min_length=8, max_length=64)
2373     version_id: str = Field(..., min_length=8, max_length=64)
2374     channel: str = Field("local_export", max_length=40)
2375     note: str = Field("", max_length=1000)
2376     artifact_path: str = Field("", max_length=300)
2377 class PublishOut(BaseModel):
2378     id: str
2379     project_id: str
2380     version_id: str
2381     channel: str
2382     note: str
2383     artifact_path: str
2384     created_at: str
2385 # apps/api/tests/__init__.py
2386 # -*- coding: utf-8 -*-
2387 # apps/api/tests/test_health.py
2388 # -*- coding: utf-8 -*-
2389 from __future__ import annotations
2390 import unittest
2391 from fastapi.testclient import TestClient
2392 from apps.api.main import create_app
2393 class HealthEndpointTests(unittest.TestCase):
2394     def test_health_ok(self) -> None:
2395         client = TestClient(create_app())
2396         resp = client.get("/health")
2397         self.assertEqual(resp.status_code, 200)
2398         self.assertEqual(resp.json().get("status"), "ok")
2399 if __name__ == "__main__":
2400     unittest.main()

```



```

2401 # apps/api/util.py
2402 # -*- coding: utf-8 -*-
2403 from __future__ import annotations
2404 import hashlib
2405 import json
2406 from datetime import datetime, timezone
2407 from typing import Any, Dict
2408 from uuid import uuid4
2409 def new_id() -> str:
2410     return uuid4().hex
2411 def utc_now() -> str:
2412     return datetime.now(tz=timezone.utc).replace(microsecond=0).isoformat()
2413 def json_dumps(obj: Any) -> str:
2414     return json.dumps(obj, ensure_ascii=False, separators=(",", ":"))
2415 def sha256_bytes(data: bytes) -> str:
2416     h = hashlib.sha256()
2417     h.update(data)
2418     return h.hexdigest()
2419 def csv_tags(tags: list[str]) -> str:
2420     clean = []
2421     for t in tags:
2422         t = (t or "").strip()
2423         if not t:
2424             continue
2425         if "," in t:
2426             t = t.replace(",", " ")
2427         clean.append(t)
2428     # de-dup, keep order
2429     seen = set()
2430     out = []
2431     for t in clean:
2432         if t in seen:
2433             continue
2434         seen.add(t)
2435         out.append(t)
2436     return ",".join(out)
2437 def parse_tags(tags_csv: str) -> list[str]:
2438     items = []
2439     for t in (tags_csv or "").split(","):
2440         t = t.strip()
2441         if t:
2442             items.append(t)
2443     return items
2444 # apps/web/admin.html
2445 <!doctype html>
2446 <html lang="zh-CN"><head><meta charset="utf-8" /><meta name="viewport"
2447 content="width=device-width, initial-scale=1" />
2448 <title>管理端 - VR 多模态交互媒介内容创作与发布系统</title></head><body>
2449 <h1>管理端</h1>
2450 <ul>

```

```

2451     <li>核心动作按钮口径：新增 / 编辑 / 查看 / 删除</li>
2452 </ul>
2453 </body></html>
2454 # apps/web/index.html
2455 <!doctype html>
2456 <html lang="zh-CN">
2457     <head>
2458         <meta charset="utf-8" />
2459         <meta name="viewport" content="width=device-width, initial-scale=1" />
2460         <title>VR 多模态交互媒介内容创作与发布系统</title>
2461     </head>
2462     <body>
2463         <h1>VR 多模态交互媒介内容创作与发布系统</h1>
2464         <ul>
2465             <li><a href="/teacher.html">业务专员端</a></li>
2466             <li><a href="/student.html">员工端</a></li>
2467             <li><a href="/admin.html">管理员端</a></li>
2468         </ul>
2469     </body>
2470 </html>
2471 # apps/web/student.html
2472 <!doctype html>
2473 <html lang="zh-CN"><head><meta charset="utf-8" /><meta name="viewport"
2474 content="width=device-width, initial-scale=1" />
2475 <title>学生端 - VR 多模态交互媒介内容创作与发布系统</title></head><body>
2476 <h1>学生端</h1>
2477 <ul>
2478     <li>当前版本：V1.0（示例）</li>
2479     <li>体验说明：按交互节点顺序体验并记录问题</li>
2480     <li>反馈方式：提交问题编号与截图（后续版本接入反馈单）</li>
2481 </ul>
2482 </body></html>
2483 # apps/web/teacher.html
2484 <!doctype html>
2485 <html lang="zh-CN"><head><meta charset="utf-8" /><meta name="viewport"
2486 content="width=device-width, initial-scale=1" />
2487 <title>教师端 - VR 多模态交互媒介内容创作与发布系统</title></head><body>
2488 <h1>教师端</h1>
2489 <ol>
2490     <li>查看项目概览与关键指标（资源数、场景数、交互数、最近发布）。</li>
2491     <li>对比两个版本的变更摘要（资源替换、交互规则变更）。</li>
2492     <li>导出发布说明与清单，用于验收留痕。</li>
2493 </ol>
2494 </body></html>
2495 # apps/api/main.py
2496 # -*- coding: utf-8 -*-
2497 from __future__ import annotations
2498 import os
2499 try:
2500     from fastapi import FastAPI

```

```

2501     from fastapi.responses import HTMLResponse
2502 except Exception: # pragma: no cover
2503     FastAPI = None # type: ignore[assignment]
2504 from apps.api.routers import (
2505     asset_router,
2506     audit_router,
2507     hr_router,
2508     interaction_router,
2509     project_router,
2510     scene_router,
2511     versioning_router,
2512 )
2513 def create_app() -> "FastAPI":
2514     if FastAPI is None:
2515         raise RuntimeError("FastAPI is not installed. Install dependencies
2516 before running the API.")
2517     app = FastAPI(title="API")
2518     app.include_router(hr_router)
2519     app.include_router(project_router)
2520     app.include_router(asset_router)
2521     app.include_router(scene_router)
2522     app.include_router(interaction_router)
2523     app.include_router(versioning_router)
2524     app.include_router(audit_router)
2525     @app.get("/", response_class=HTMLResponse)
2526     def index() -> str:
2527         return """<!doctype html>
2528 <html lang="zh-CN">
2529 <head>
2530 <meta charset="utf-8" />
2531 <meta name="viewport" content="width=device-width, initial-scale=1" />
2532 <title>VR 多模态交互媒介内容创作与发布系统</title>
2533 <style>
2534     :root { --bg:#0b1020; --panel:#121a33; --text:#e8ecff; --muted:#aab3d9; -
2535 -accent:#6ee7ff; --danger:#ff5b6e; }
2536     body { margin:0; font-family: "Segoe UI", "Microsoft YaHei", sans-serif;
2537 background:linear-gradient(180deg,#070a14, var(--bg)); color:var(--text); }
2538     header { display:flex; align-items:center; justify-content:space-between;
2539 padding:14px 18px; background:rgba(18,26,51,.8); border-bottom:1px solid
2540 rgba(110,231,255,.18); position:sticky; top:0; backdrop-filter: blur(8px); }
2541     header .brand { font-weight:700; letter-spacing:.5px; }
2542     header .badge { font-size:12px; padding:4px 10px; border:1px solid
2543 rgba(110,231,255,.35); border-radius:999px; color:var(--accent); }
2544     main { max-width:1180px; margin:18px auto; padding:0 18px 28px;
2545 display:grid; grid-template-columns: 260px 1fr; gap:16px; }
2546     nav { background:rgba(18,26,51,.7); border:1px solid rgba(110,231,255,
2547 .12); border-radius:14px; padding:12px; }
2548     nav a { display:block; color:var(--muted); text-decoration:none;
2549 padding:10px 10px; border-radius:10px; }
2550     nav a:hover { background:rgba(110,231,255,.08); color:var(--text); }

```

```

2551     .panel { background:rgba(18,26,51,.7); border:1px solid rgba(110,231,255,
2552 .12); border-radius:14px; padding:14px; }
2553     .grid { display:grid; grid-template-columns: repeat(3, 1fr); gap:12px; }
2554     .card { border:1px solid rgba(110,231,255,.12); border-radius:12px;
2555 padding:12px; background:rgba(9,12,24,.55); }
2556     .card h3 { margin:0 0 8px; font-size:14px; color:var(--accent); }
2557     .hint { color:var(--muted); font-size:12px; line-height:1.6; }
2558     .toolbar { display:flex; gap:8px; flex-wrap:wrap; margin:8px 0 6px; }
2559     button { cursor:pointer; border-radius:10px; border:1px solid rgba(110,
2560 231,255,.22); background:rgba(110,231,255,.10); color:var(--text);
2561 padding:8px 12px; }
2562     button:hover { background:rgba(110,231,255,.16); }
2563     button.danger { border-color:rgba(255,91,110,.35); background:rgba(255,
2564 91,110,.12); }
2565     .k { font-family: ui-monospace, SFMono-Regular, Menlo, Consolas,
2566 monospace; color:var(--accent); }
2567     .hidden { display:none !important; }
2568     #loginBox { max-width:520px; margin:10vh auto; padding:18px; }
2569     #loginBox .row { display:grid; gap:10px; margin-top:10px; }
2570     input { padding:10px 12px; border-radius:10px; border:1px solid rgba(110,
2571 231,255,.18); background:rgba(255,255,255,.05); color:var(--text); }
2572     .row label { color:var(--muted); font-size:12px; }
2573     .row small { color:var(--muted); }
2574     #modal.modal-shell.modal { position:fixed; inset:0; background:rgba(0,0,
2575 0,.55); display:flex; align-items:center; justify-content:center; }
2576     #modal .dialog { width:min(680px, 92vw); border-radius:14px;
2577 background:rgba(18,26,51,.96); border:1px solid rgba(110,231,255,.18);
2578 padding:14px; box-shadow:0 24px 80px rgba(0,0,0,.45); }
2579     #modal .dialog header { position:static; border:none;
2580 background:transparent; padding:0 0 10px; }
2581     #modal .dialog .title { font-weight:700; }
2582     #modal .dialog .content { color:var(--muted); line-height:1.7; font-
2583 size:13px; }
2584 </style>
2585 </head>
2586 <body>
2587 <div id="loginView">
2588 <div id="loginBox" class="panel">
2589 <h1 style="margin:0 0 8px; font-size:18px;
2590 ">VR 多模态交互媒介内容创作与发布系统</h1>
2591 <div class="hint">V1.0 演示环境：用于截图与材料生成的最小可运行界面。<
2592 /div>
2593 <form id="login-form" class="row" onsubmit="return
2594 window.__app.login(event)">
2595 <label>用户名 <input name="username" value="admin" autocomplete=
2596 "username" /></label>
2597 <label>密码 <input name="password" type="password" value="admin123"
2598 autocomplete="current-password" /></label>
2599 <button type="submit">登录</button>
2600 <small>提示：输入任意值均可登录（用于演示与截图）。</small>

```

```

2601     </form>
2602 </div>
2603 </div>
2604 <div id="appView" class="hidden">
2605     <header>
2606         <div class="badge">V1.0</div>
2607     </header>
2608     <main>
2609         <nav>
2610             <a href="/#home">首页</a>
2611             <a href="/#zones">资源分区</a>
2612             <a href="/#stations">工作台（Stations）</a>
2613             <a href="/#inspections">交互检查</a>
2614             <a href="/#alerts">风险提醒</a>
2615             <a href="/#irrigation">发布流水</a>
2616             <a href="/#reports">统计报表</a>
2617         </nav>
2618         <section class="panel">
2619             <div id="pageTitle" style="font-weight:700; margin-bottom:10px;
2620 ">首页</div>
2621             <div id="pageBody"></div>
2622         </section>
2623     </main>
2624 </div>
2625 <div id="modal" class="modal-shell modal hidden">
2626     <div class="dialog" role="dialog" aria-modal="true">
2627         <header>
2628             <div class="title" id="modalTitle">操作</div>
2629             <button class="danger" onclick="window.__rkModal.closeModal()
2630 ">关闭</button>
2631         </header>
2632         <div class="content" id="modalContent"></div>
2633     </div>
2634 </div>
2635 <script>
2636 window.__rkModal = {
2637     closeModal() {
2638         document.getElementById('modal').classList.add('hidden');
2639     },
2640     openModal(title, content) {
2641         document.getElementById('modalTitle').textContent = title;
2642         document.getElementById('modalContent').textContent = content;
2643         document.getElementById('modal').classList.remove('hidden');
2644     }
2645 };
2646 function renderHome() {
2647     document.getElementById('pageTitle').textContent = '首页';
2648     document.getElementById('pageBody').innerHTML = `
2649     <div class="grid">
2650         <div class="card"><h3>项目概览</h3><div class=

```

```

2651 "hint">资源、场景、交互与发布记录在此汇总展示。</div></div>
2652 </div>`;
2653 }
2654 function renderSimplePage(title, id) {
2655   document.getElementById('pageTitle').textContent = title;
2656   const body = document.getElementById('pageBody');
2657   body.innerHTML = `<div id="${id}" class="card">
2658     <h3>${title}</h3>
2659   </div>`;
2660 }
2661 function renderStations() {
2662   document.getElementById('pageTitle').textContent =
2663 '工作台（Stations）';
2664   const body = document.getElementById('pageBody');
2665   body.innerHTML = `
2666     <div id="stations" class="card">
2667       <h3>Stations 列表</h3>
2668       <div class="hint">用于演示“新增/编辑/查看/删除”弹窗操作按钮文案。<
2669 /div>
2670       <div class="toolbar">
2671         <button onclick="window.__rkModal.openModal('新增',
2672 '创建新的条目并填写字段。')">新增</button>
2673         <button onclick="window.__rkModal.openModal('编辑',
2674 '修改条目名称、标签与说明。')">编辑</button>
2675         <button onclick="window.__rkModal.openModal('查看',
2676 '查看条目详情与引用关系。')">查看</button>
2677         <button class="danger" onclick="window.__rkModal.openModal('删除
2678 ', '删除条目前请确认无引用并记录审计。')">删除</button>
2679       </div>
2680       <div class="hint">提示：截图脚本会自动点击以上按钮并截取弹窗状态。
2681 </div>
2682     </div>`;
2683 }
2684 function route() {
2685   const hash = (location.hash || '#home').replace(/^#/ , "");
2686   if (hash === 'home' || hash === "") return renderHome();
2687   if (hash === 'stations') return renderStations();
2688   if (hash === 'zones') return renderSimplePage('资源分区', 'zones');
2689   if (hash === 'inspections') return renderSimplePage('交互检查',
2690 'inspections');
2691   if (hash === 'alerts') return renderSimplePage('风险提醒', 'alerts');
2692   if (hash === 'irrigation') return renderSimplePage('发布流水',
2693 'irrigation');
2694   if (hash === 'reports') return renderSimplePage('统计报表', 'reports');
2695   return renderSimplePage('页面', hash);
2696 }
2697 window.__app = {
2698   login(e) {
2699     e.preventDefault();
2700     localStorage.setItem('__demo_logged_in', '1');

```

```

2701     document.getElementById('loginView').classList.add('hidden');
2702     document.getElementById('appView').classList.remove('hidden');
2703     route();
2704     return false;
2705 }
2706 };
2707 window.addEventListener('hashchange', () => route());
2708 if (localStorage.getItem('__demo_logged_in') === '1') {
2709     document.getElementById('loginView').classList.add('hidden');
2710     document.getElementById('appView').classList.remove('hidden');
2711     route();
2712 }
2713 </script>
2714 </body>
2715 </html>""
2716 @app.get("/health")
2717 def health() -> dict[str, str]:
2718     return {"status": "ok"}
2719 return app
2720 app = create_app()
2721 def _main() -> None:
2722     import uvicorn
2723     port = int(os.environ.get("APP_PORT", "8010"))
2724     uvicorn.run("apps.api.main:app", host="127.0.0.1", port=port, log_level=
2725 "warning")
2726 if __name__ == "__main__":
2727     _main()
2728 # apps/api/routers/_init_.py
2729 # -*- coding: utf-8 -*-
2730 from __future__ import annotations
2731 from .hr import router as hr_router
2732 from .project import router as project_router
2733 from .asset import router as asset_router
2734 from .scene import router as scene_router
2735 from .interaction import router as interaction_router
2736 from .versioning import router as versioning_router
2737 from .audit import router as audit_router
2738 __all__ = [
2739     "hr_router",
2740     "project_router",
2741     "asset_router",
2742     "scene_router",
2743     "interaction_router",
2744     "versioning_router",
2745     "audit_router",
2746 ]
2747 # apps/api/routers/asset.py
2748 # -*- coding: utf-8 -*-
2749 from __future__ import annotations
2750 from fastapi import APIRouter, HTTPException

```

```

2751 from apps.api.schemas.asset import AssetCreate, AssetOut
2752 from apps.api.services import asset_repo
2753 router = APIRouter(prefix="/api/assets", tags=["assets"])
2754 @router.get("", response_model=list[AssetOut])
2755 def list_assets(project_id: str):
2756     return asset_repo.list_assets(project_id)
2757 @router.post("", response_model=dict)
2758 def create_asset(payload: AssetCreate):
2759     aid = asset_repo.create_asset(
2760         payload.project_id,
2761         payload.type,
2762         payload.filename,
2763         payload.sha256,
2764         payload.size,
2765         payload.tags,
2766         payload.meta_json,
2767     )
2768     return {"id": aid}
2769 @router.delete("/{asset_id}", response_model=dict)
2770 def delete_asset(asset_id: str):
2771     ok = asset_repo.delete_asset(asset_id)
2772     if not ok:
2773         raise HTTPException(status_code=404, detail="not found")
2774     return {"deleted": True}
2775 # apps/api/routers/audit.py
2776 # -*- coding: utf-8 -*-
2777 from __future__ import annotations
2778 from fastapi import APIRouter
2779 from apps.api.services import audit_repo
2780 router = APIRouter(prefix="/api/audit", tags=["audit"])
2781 @router.get("", response_model=list[dict])
2782 def list_audit(limit: int = 100):
2783     return audit_repo.list_audit(limit=limit)
2784 # apps/api/routers/hr.py
2785 # -*- coding: utf-8 -*-
2786 from __future__ import annotations
2787 from fastapi import APIRouter, HTTPException
2788 from pydantic import BaseModel, Field
2789 from apps.api.services.store import InMemoryStore
2790 router = APIRouter(prefix="/api/hr", tags=["hr"])
2791 class DepartmentIn(BaseModel):
2792     name: str = Field(..., min_length=2, max_length=50)
2793 class PositionIn(BaseModel):
2794     name: str = Field(..., min_length=2, max_length=50)
2795     dept_id: str
2796 class EmployeeIn(BaseModel):
2797     name: str = Field(..., min_length=2, max_length=50)
2798     dept_id: str
2799     title: str = Field(..., min_length=1, max_length=50)
2800 def get_store() -> InMemoryStore:

```



```

2801     # module-level singleton to keep the example minimal.
2802     global _STORE
2803     try:
2804         return _STORE
2805     except NameError:
2806         _STORE = InMemoryStore()
2807     return _STORE
2808 @router.get("/departments")
2809 def list_departments() -> list[dict]:
2810     return get_store().list_departments()
2811 @router.post("/departments")
2812 def create_department(payload: DepartmentIn) -> dict:
2813     _id = get_store().create_department(payload.name)
2814     return {"id": _id}
2815 @router.delete("/departments/{dept_id}")
2816 def delete_department(dept_id: str) -> dict:
2817     ok = get_store().delete_department(dept_id)
2818     if not ok:
2819         raise HTTPException(status_code=404, detail="not found")
2820     return {"deleted": True}
2821 @router.get("/positions")
2822 def list_positions() -> list[dict]:
2823     return get_store().list_positions()
2824 @router.post("/positions")
2825 def create_position(payload: PositionIn) -> dict:
2826     _id = get_store().create_position(payload.name, payload.dept_id)
2827     return {"id": _id}
2828 @router.delete("/positions/{pos_id}")
2829 def delete_position(pos_id: str) -> dict:
2830     ok = get_store().delete_position(pos_id)
2831     if not ok:
2832         raise HTTPException(status_code=404, detail="not found")
2833     return {"deleted": True}
2834 @router.get("/employees")
2835 def list_employees() -> list[dict]:
2836     return get_store().list_employees()
2837 @router.post("/employees")
2838 def create_employee(payload: EmployeeIn) -> dict:
2839     _id = get_store().create_employee(payload.name, payload.dept_id,
2840     payload.title)
2841     return {"id": _id}
2842 @router.delete("/employees/{emp_id}")
2843 def delete_employee(emp_id: str) -> dict:
2844     ok = get_store().delete_employee(emp_id)
2845     if not ok:
2846         raise HTTPException(status_code=404, detail="not found")
2847     return {"deleted": True}
2848 # apps/api/routers/interaction.py
2849 # -*- coding: utf-8 -*-
2850 from __future__ import annotations

```

```

2851 from fastapi import APIRouter
2852 from apps.api.schemas.interaction import InteractionCreate, InteractionOut
2853 from apps.api.services import interaction_repo
2854 router = APIRouter(prefix="/api/interactions", tags=["interactions"])
2855 @router.post("", response_model=dict)
2856 def create_interaction(payload: InteractionCreate):
2857     iid = interaction_repo.create_interaction(payload.project_id,
2858     payload.scene_id, payload.name, payload.enabled, payload.graph_json)
2859     return {"id": iid}
2860 @router.get("", response_model=list[InteractionOut])
2861 def list_interactions(project_id: str):
2862     return interaction_repo.list_interactions(project_id)
2863 # apps/api/routers/project.py
2864 # -*- coding: utf-8 -*-
2865 from __future__ import annotations
2866 from fastapi import APIRouter, HTTPException
2867 from apps.api.schemas.project import ProjectCreate, ProjectOut,
2868 ProjectUpdate
2869 from apps.api.services import project_repo
2870 router = APIRouter(prefix="/api/projects", tags=["projects"])
2871 @router.get("", response_model=list[ProjectOut])
2872 def list_projects():
2873     return project_repo.list_projects()
2874 @router.post("", response_model=dict)
2875 def create_project(payload: ProjectCreate):
2876     pid = project_repo.create_project(payload.name, payload.target_device,
2877     payload.constraints_json)
2878     return {"id": pid}
2879 @router.get("/{project_id}", response_model=ProjectOut)
2880 def get_project(project_id: str):
2881     p = project_repo.get_project(project_id)
2882     if not p:
2883         raise HTTPException(status_code=404, detail="not found")
2884     return p
2885 @router.patch("/{project_id}", response_model=dict)
2886 def patch_project(project_id: str, payload: ProjectUpdate):
2887     ok = project_repo.update_project(project_id, payload.model_dump())
2888     if not ok:
2889         raise HTTPException(status_code=404, detail="not found or no
2890     changes")
2891     return {"updated": True}
2892 @router.delete("/{project_id}", response_model=dict)
2893 def delete_project(project_id: str):
2894     ok = project_repo.delete_project(project_id)
2895     if not ok:
2896         raise HTTPException(status_code=404, detail="not found")
2897     return {"deleted": True}
2898 # apps/api/routers/publish.py
2899 # apps/api/routers/scene.py
2900 # -*- coding: utf-8 -*-

```

```

2901 from __future__ import annotations
2902 from fastapi import APIRouter, HTTPException
2903 from apps.api.schemas.scene import SceneCreate, SceneNodeCreate,
2904 SceneNodeOut, SceneOut
2905 from apps.api.services import scene_repo
2906 router = APIRouter(prefix="/api/scenes", tags=["scenes"])
2907 @router.post("", response_model=dict)
2908 def create_scene(payload: SceneCreate):
2909     sid = scene_repo.create_scene(payload.project_id, payload.name)
2910     return {"id": sid}
2911 @router.get("", response_model=list[SceneOut])
2912 def list_scenes(project_id: str):
2913     return scene_repo.list_scenes(project_id)
2914 @router.post("/nodes", response_model=dict)
2915 def create_node(payload: SceneNodeCreate):
2916     nid = scene_repo.create_node(
2917         payload.scene_id,
2918         payload.parent_id,
2919         payload.name,
2920         payload.node_type,
2921         payload.transform_json,
2922         payload.asset_ref_id,
2923         payload.props_json,
2924     )
2925     return {"id": nid}
2926 @router.get("/{scene_id}/nodes", response_model=list[SceneNodeOut])
2927 def list_nodes(scene_id: str):
2928     return scene_repo.list_nodes(scene_id)
2929 # apps/api/routers/versioning.py
2930 # -*- coding: utf-8 -*-
2931 from __future__ import annotations
2932 from fastapi import APIRouter
2933 from apps.api.schemas.versioning import PublishCreate, PublishOut,
2934 VersionCreate, VersionOut
2935 from apps.api.services import publish_repo, versioning_repo
2936 router = APIRouter(prefix="/api/versioning", tags=["versioning"])
2937 @router.post("/versions", response_model=dict)
2938 def create_version(payload: VersionCreate):
2939     vid = versioning_repo.create_version(payload.project_id, payload.version,
2940     payload.note)
2941     return {"id": vid}
2942 @router.get("/versions", response_model=list[VersionOut])
2943 def list_versions(project_id: str):
2944     return versioning_repo.list_versions(project_id)
2945 @router.post("/publish", response_model=dict)
2946 def create_publish(payload: PublishCreate):
2947     pid = publish_repo.create_publish_record(payload.project_id,
2948     payload.version_id, payload.channel, payload.note, payload.artifact_path)
2949     return {"id": pid}
2950 @router.get("/publish", response_model=list[PublishOut])

```

```

2951 def list_publish(project_id: str):
2952     return publish_repo.list_publish_records(project_id)
2953 # apps/api/domain/__init__.py
2954 # apps/api/domain/models.py
2955 # -*- coding: utf-8 -*-
2956 from __future__ import annotations
2957 from dataclasses import dataclass
2958 from datetime import datetime
2959 @dataclass(frozen=True)
2960 class Department:
2961     id: str
2962     name: str
2963     created_at: datetime
2964 @dataclass(frozen=True)
2965 class Position:
2966     id: str
2967     name: str
2968     dept_id: str
2969     created_at: datetime
2970 @dataclass(frozen=True)
2971 class Employee:
2972     id: str
2973     name: str
2974     dept_id: str
2975     title: str
2976     created_at: datetime
2977 # apps/api/services/__init__.py
2978 # apps/api/services/asset_repo.py
2979 # -*- coding: utf-8 -*-
2980 from __future__ import annotations
2981 import sqlite3
2982 from apps.api.db import db
2983 from apps.api.util import csv_tags, new_id, parse_tags, utc_now
2984 def create_asset(project_id: str, type: str, filename: str, sha256: str,
2985 size: int, tags: list[str], meta_json: str) -> str:
2986     aid = new_id()
2987     with db() as conn:
2988         conn.execute(
2989             "INSERT INTO asset(id,project_id,type,filename,sha256,size,tags,
2990 meta_json,created_at) VALUES (?,?,?,?,?,?,?,?,?)",
2991             (aid, project_id, type, filename, sha256, int(size),
2992 csv_tags(tags), meta_json or "{}", utc_now()),
2993         )
2994     return aid
2995 def list_assets(project_id: str) -> list[dict]:
2996     with db() as conn:
2997         rows = conn.execute("SELECT * FROM asset WHERE project_id=? ORDER BY
2998 created_at DESC", (project_id,)).fetchall()
2999         out = []
3000         for r in rows:

```